

2022 年度

修 士 論 文

PowerShell を用いた
ファイルレス攻撃に対する
アーティファクト保存手法の
提案と実装

指導教員: 上原 哲太郎

立命館大学大学院 情報理工学研究科
博士課程前期課程 情報理工学専攻
計算機科学コース

学生証番号: 6611210032-7

氏名: 木村 将人

概要

多くの攻撃者は、標的マシンにファイルを生成せずに攻撃を行うファイルレス攻撃において、PowerShell を利用する。PowerShell を用いたファイルレス攻撃によるインシデントが発生した場合、その調査手法として主にログ解析が用いられる。PowerShell の操作に関わるログは、PowerShell に標準で備わるログシステムによって、Windows イベントログの 1 つとして記録される。ログ解析を確実に行うためには、PowerShell ログシステムによって記録されるログを保護する必要がある。一方で、多くの攻撃者は、攻撃に関わるアーティファクトを標的マシンに極力残さないアンチ・フォレンジック手法を攻撃に組み込む。そのため、既存の PowerShell ログシステムにおいて、ログ解析に必要なログを確実に保全できない可能性がある。そこで、本研究では PowerShell を用いたファイルレス攻撃を想定し、PowerShell 内部に独自ログシステムを組み込み、直接ローカル外ログサーバにログを送信する手法を提案する。本提案手法によって、PowerShell を用いたファイルレス攻撃において、アンチ・フォレンジック手法を組み込まれたとしても、ログを確実に保全することを目指す。

目次

第 1 章	はじめに	3
第 2 章	研究背景	5
2.1	サイバーセキュリティの現状	5
2.2	ファイルレス攻撃	6
2.3	PowerShell	8
第 3 章	準備	12
3.1	コンピュータセキュリティインシデント	12
3.2	デジタル・フォレンジック	13
3.3	コンピュータセキュリティログ	15
3.4	PowerShell ログシステム	16
3.5	PowerShell ログシステムとアンチ・フォレンジック	20
第 4 章	関連研究	24
4.1	ログの保全に関する関連研究	24
4.2	関連研究と PowerShell ログシステム	25
第 5 章	提案手法	28
5.1	概要	28
5.2	独自保全システムの全体像	29
5.3	保全のための要件	30
5.4	独自保全システムの実現	30
5.5	難読化への対応	33
第 6 章	実装	34
6.1	実装環境	34
6.2	独自ログシステムの実装	35
6.3	ログサーバの機能	39
第 7 章	評価	43

7.1	概要	43
7.2	独自ログシステムの性能	43
7.3	独自保全システムにおけるログ保全の可否	44
7.4	独自ログシステムの難読化解除	47
第 8 章	考察と今後の課題	49
8.1	本評価に対する考察	49
8.2	今後の課題	50
第 9 章	おわりに	52
参考文献		54

第 1 章

はじめに

サイバー攻撃では、標的マシン上で悪意のあるファイルを用いて攻撃を行うファイルベース攻撃が主流である。一方で、標的マシンにファイルを生成せずに攻撃を行うファイルレス攻撃の登場により、サイバー攻撃への対応が更に困難になっている。Center for Internet Security が 2022 年 8 月に発表した「Only in Memory: Fileless Malware - An Elusive TTP」[1]によると、サイバー攻撃全体に対するファイルレス攻撃の割合は年々増加しており、2022 年では 50% を占めると推測されている。多くの攻撃者はファイルレス攻撃を行った際に、標的マシン上に標準で組み込まれている正規ツールを利用する。Kaspersky は、2019 年に最もサイバー攻撃の中で利用された正規ツールが PowerShell と報告した [2]。PowerShell とは、Windows に標準で搭載されているコマンドラインシェルおよびスクリプト言語である。PowerShell における実行は、ファイルを生成せずにメインメモリ上で行われる。PowerShell には、システム管理やタスク自動化に利用できる数千のコマンドが搭載されている。PowerShell を用いることで、レジストリや Windows API といった Windows システム内にある機能の大半を使用できる。PowerShell は高機能なツールである反面、攻撃者に狙われる。さらに、Windows 標準の正規ツールである PowerShell では、実行されたコマンドやスクリプトが正規のものか攻撃のものかの判別が難しい。そのため、攻撃者は、PowerShell を用いたファイルレス攻撃を行うことで、高度でステルス性の高い攻撃ができる。

コンピュータを対象としたサイバー攻撃によるインシデントが発生した際、原因究明のためにデジタル・フォレンジックを用いて、コンピュータに保存されたデータの保全や解析が行われる。デジタル・フォレンジックを用いたインシデント調査では、主にコンピュータの補助記憶装置を対象として分析が進められる。一方で、ファイルレス攻撃のような補助記憶装置にファイルを生成しない攻撃においては、従来の補助記憶装置を対象としたデジタル・フォレンジックで対応できない。そのため、PowerShell を用いたファイルレス攻撃によるインシデントに対し、主な調査手法としてメモリ・フォレンジックもしくは、ログ解析が用いられる。メモリ・フォレンジックは、主記憶装置（メインメモリ）を対象とした解析手法であり、ファイルレス攻撃によるインシデント調査に有効である。しかし、メインメモリは揮発性が高く、コンピュータの電源供給を絶つことでデータが消失してしまうため、すべてのインシデント調査でメモリ・フォレンジックによる解析が実施できる保証はない。そこで、一般的に PowerShell を用いたファイルレス攻撃によるインシデントが

発生した際、ログ解析が行われる。

PowerShell には、操作に関わるログを記録するシステム（以下、「PowerShell ログシステム」とする）が標準で搭載されている。インシデントが発生した際、原因究明のために PowerShell ログシステムで記録されるログ（以下、「PowerShell ログ」とする）を保全し、解析を行う。一方で、PowerShell ログを標的マシン上に残さないようなテクニックを組み込んだ攻撃が確認されている。このような、デジタル・フォレンジックの調査に必要なアーティファクトを標的マシン上に残さない手法をアンチ・フォレンジックという。PowerShell ログシステムに対するアンチ・フォレンジック手法として、「PowerShell ログシステムの無効化」、「PowerShell ログの改ざん・削除」および「PowerShell スクリプトの難読化」がある。PowerShell を用いたファイルレス攻撃において、アンチ・フォレンジック手法が組み込まれると、標準の PowerShell ログシステムでは調査に必要なログを保全できない。そこで、本研究では PowerShell 内部に独自ログシステムを組み込み、直接ローカル外ログサーバ（以下、「ログサーバ」とする）にログを保全する手法を提案する。本研究によって、PowerShell を用いたファイルレス攻撃によるインシデントが発生した際、攻撃にアンチ・フォレンジック手法が組み込まれていても、PowerShell ログを確実に保全することを目指す。

本論文の構成は以下の通りである。2 章の研究背景で、PowerShell とファイルレス攻撃の関係について説明する。3 章では準備として、PowerShell を用いたファイルレス攻撃に対するアンチ・フォレンジック手法について技術的な背景を中心に説明する。4 章では、本研究の関連研究を説明する。5 章では提案手法として、本研究で提案する PowerShell 内部への独自ログシステム組み込みについて説明する。6 章では実装として、5 章で提案した手法に基づいた実装の詳細について説明する。7 章では評価として、提案手法に基づいて実装したシステムの評価を行う。8 章では考察として、7 章の評価に対する提案手法の実用性について論じる。9 章では、7 章の評価に対する結論と今後の課題の整理を行う。最後に、10 章で本研究に対するまとめを述べる。

第2章

研究背景

2.1 サイバーセキュリティの現状

2.1.1 サイバーセキュリティ上の脅威の増大

総務省が2022年に発表した「令和4年版 情報通信白書」により、NICTERにおけるサイバー攻撃関連の通信数の推移を図2.1に示す。本資料によると、2021年のサイバー攻撃関連の通信数が2018年と比べて2.4倍になっている。このことから、サイバーセキュリティ上の脅威が今後も増大していくと考えられる。

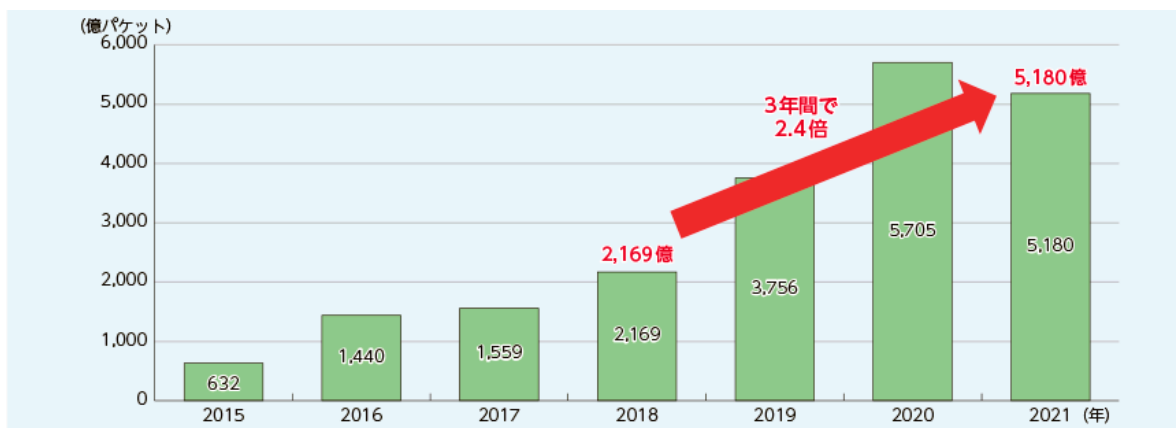


図 2.1 NICTER におけるサイバー攻撃関連の通信数の推移 [3]

2.1.2 サイバー攻撃の巧妙化

独立行政法人情報処理推進機構が2022年に発表した「情報セキュリティ 10 大脅威 2023」[4]によると、組織における脅威の1位、3位がそれぞれ「ランサムウェアによる被害」、「標的型攻撃による機密情報の窃取」であった。ランサムウェアや標的型攻撃で使用するマルウェアには、実行

時にファイルを利用しないファイルレス技術や難読化といったセキュリティ対策ソフトの検知を回避するテクニックが組み込まれているものもある。2017 年 4 月に、ファイルレス技術を組み込んだランサムウェアの SOREBRECT、2021 年には、ファイルレス型のバックドア機能を持つマルウェアの LODEINFO が確認されている。これらのマルウェアには、セキュリティ対策ソフトの検知を回避する機能のみならず、攻撃に関わる痕跡を残さないような機能が搭載されている。そのため、攻撃に関わる情報を十分に収集できず、攻撃の全体像の把握が年々困難になっている。

2.2 ファイルレス攻撃

2.2.1 ファイルベース攻撃とファイルレス攻撃

ファイルベース攻撃とは、標的マシンにバイナリファイルやスクリプトファイルをはじめとする攻撃用のファイルを、生成もしくはダウンロードして実行する攻撃である。対して、ファイルレス攻撃とは、標的マシンにファイルを生成しない攻撃である。従来のアンチ・ウイルスソフトでは、主にマシン上のファイルを検査するため、ファイルベース攻撃の検知ができていてもファイルレス攻撃を検知できない。ファイルレス攻撃では、ファイルを生成して実行しない代わりに、PowerShell や Microsoft Office のマクロのような標的マシンに組み込まれている正規ツールを悪用する。そのため、アンチ・ウイルスソフトでは、使用された正規ツールが通常利用のものの攻撃利用のものを判別することが難しい。

2.2.2 ファイルレス攻撃の動向

Center for Internet Security が 2022 年 8 月に発表した「Only in Memory: Fileless Malware - An Elusive TTP」[1]における、ファイルレス攻撃頻度の予測を図 2.2 に示す。2016 年はサイバー攻撃の 20% がファイルレス攻撃であったのに対し、2019 年には 35% に増加した。2022 年には 50% まで増加していると見込まれており、その後もサイバー攻撃に対するファイルレス攻撃の割合が増加すると予測されている。

アンラボ社が 2020 年 10 月に発表した「ハッカーの強力な武器、ファイルレス攻撃とは」における、ファイルレス攻撃の登場と増加の変化を図 2.3 に示す。2000 年代からメモリ常駐型マルウェアを利用したファイルレス攻撃が存在していた。ファイルレス攻撃の事例として、2014 年に登場した「Emotet」が挙げられる。2016 年以降は、サイバー攻撃において PowerShell が利用されるようになり、ファイルレス攻撃は増加傾向を示した。そして、2018 年から PowerShell を用いたファイルレス攻撃によるインシデントが確認され始めた。このことから、ファイルレス攻撃は今後も増加すると考えられる。

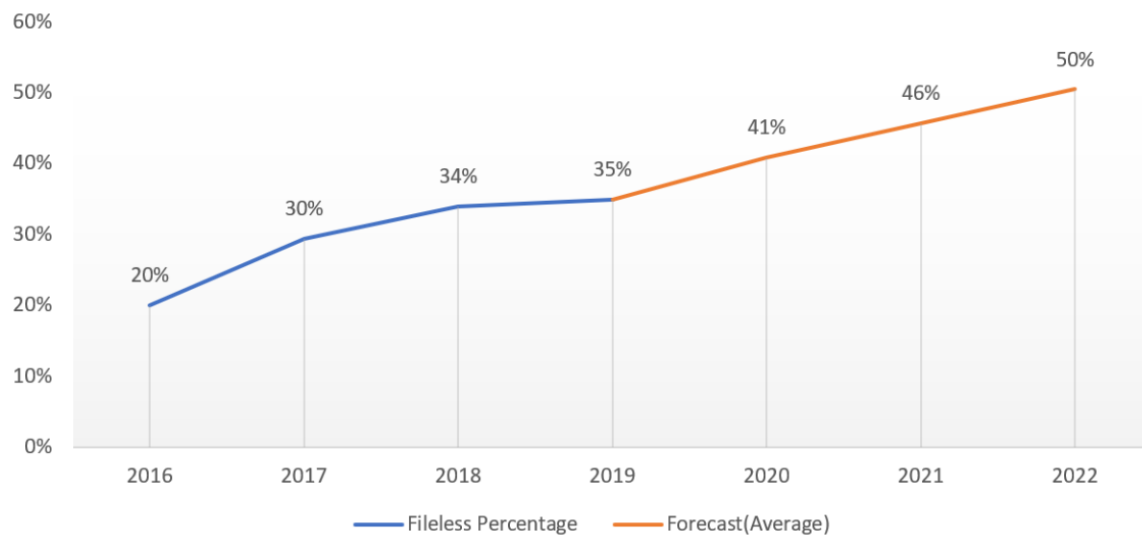


図 2.2 Ponemon Institute が 2020 年度に発表した「The Third Annual Study on the State of Endpoint Security Risk」に基づくファイルレス攻撃頻度の予測 [1]

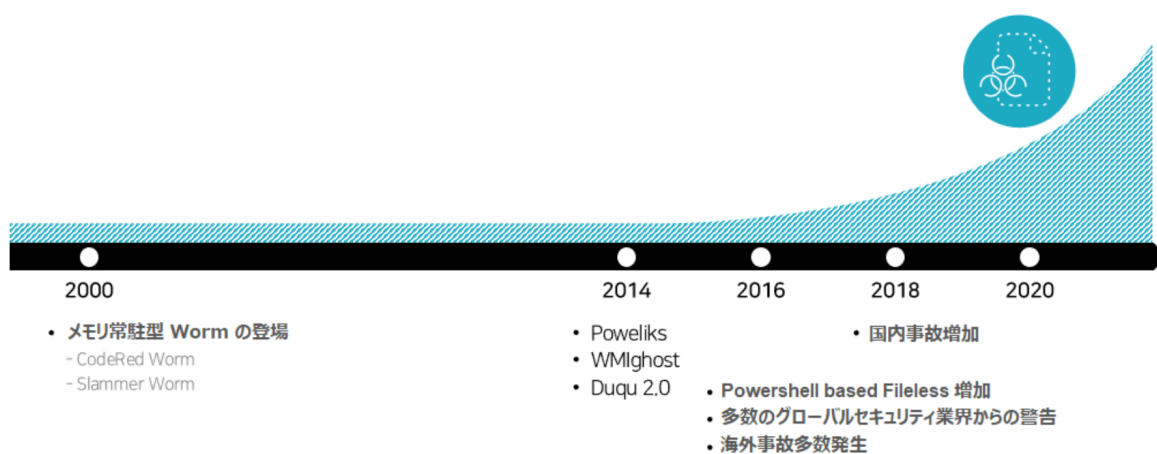


図 2.3 ファイルレス攻撃の変化 [5]

2.3 PowerShell

2.3.1 概要

PowerShell とは、Microsoft 社が開発する“コマンドラインシェル、スクリプト言語、および構成管理フレームワークで構成されるクロスプラットフォームのタスク自動化ソリューション”である [6]。PowerShell は、Windows に標準搭載されており、システム管理やタスク自動化に利用できる数千のコマンドを持つ。PowerShell で扱われるコマンドは、「動詞-名詞」の命名規則があり、「コマンドレット」と呼ばれている。PowerShell を用いることで、Windows システム内にある機能の大半を使用できる。PowerShell で利用できる Windows システム内にある機能の具体例として、「レジストリの操作」、「Windows API の利用」、「ファイルシステムの操作」、「WinRM (Windows Remote Management) を介したリモート操作」が挙げられる。PowerShell における実行は、ファイルを生成せずにメインメモリ上で行われる。

2.3.2 変遷

PowerShell は、2023 年 1 月時点で「Windows PowerShell」と「PowerShell Core」に大別される。表 2.1 に示すように、Windows 10 および 2023 年 1 月時点の現行 Windows 11 に標準搭載されている PowerShell は、「Windows PowerShell 5.1」である。本論文では、PowerShell Core 6.0 以降の PowerShell を「PowerShell 7」と呼ぶこととする。PowerShell 7 では、OSS（オープンソース・ソフトウェア）となりマルチプラットフォーム化を果たした。つまり、Windows 以外の macOS や Linux でも PowerShell を利用できる。現在、Windows においては、PowerShell 7 を別途インストールすることで、Windows PowerShell 5.1 と PowerShell 7 を併用できる。PowerShell 7 は、Windows PowerShell 5.1 の基本的な部分を引き継ぎ、Windows 以外のプラットフォームに対応するため一部機能を強化している。Microsoft の PowerShell 開発チームは、今後、“Windows PowerShell に新しい機能を導入する予定はない”と発表している [7]。今後は PowerShell 7 が積極的にバージョンアップされ、将来的に Windows PowerShell から PowerShell 7 への完全な移行が想定される。

表 2.1 PowerShell の変遷

リリース	名称	搭載 OS
2006 年 11 月	Windows PowerShell 1.0	Windows Vista
2009 年 10 月	Windows PowerShell 2.0	Windows 7
2012 年 9 月	Windows PowerShell 3.0	Windows 8
2013 年 10 月	Windows PowerShell 4.0	Windows 8.1
2016 年 2 月	Windows PowerShell 5.0	Windows 10 (Ver.1511)
2016 年 8 月	Windows PowerShell 5.1	Windows 10 (Ver.1607)・11
2016 年 8 月	PowerShell Core 6.0*	Windows 8.1・10, macOS, Linux
2020 年 3 月	PowerShell 7.0*	Windows 8.1・10, macOS, Linux
2022 年 11 月	PowerShell 7.3*	Windows 8.1・10・11, macOS, Linux

* 標準では非搭載のため、別途インストールが必要。

2.3.3 PowerShell とファイルレス攻撃

Kaspersky が 2019 年に発表した「Incident Response Analyst Report 2019」において、同社グローバル緊急対応チーム（GERT: Global Emergency Response Team）が 2019 年にインシデント対応したサイバー攻撃の中で [2], 使用された正規ツールの割合上位を図 2.4 に示す。この分析によると、PowerShell が最も使用された正規ツールであることがわかる。また、正規ツールである PowerShell において、実行されたコマンドやスクリプトが正規のものか攻撃のものかの判別が難しい。そのため、多くの攻撃者はファイルレス攻撃において PowerShell を利用すると考えられる。

2.3.4 ファイルレス攻撃における PowerShell の悪用事例

ファイルレス攻撃における PowerShell の悪用事例の一つとして、2015 年にサイバー犯罪集団「Pawn Storm」が米国民党全国委員会から機密情報を窃取した事例がある。攻撃グループは、Windows システムの構成要素の情報を取得し管理できる WMI（Windows Management Instrumentation）に PowerShell スクリプトを登録し、標的マシンにバックドアを設置後、機密情報を窃取した。また、2019 年に三菱電機株式会社が、第三者による不正アクセスを受けたことで、個人情報および企業機密情報が外部流出した可能性があることを報告した。この事例において、攻撃者は PowerShell に備わる遠隔操作機能を用いて、データ窃取や感染拡大を試みた。

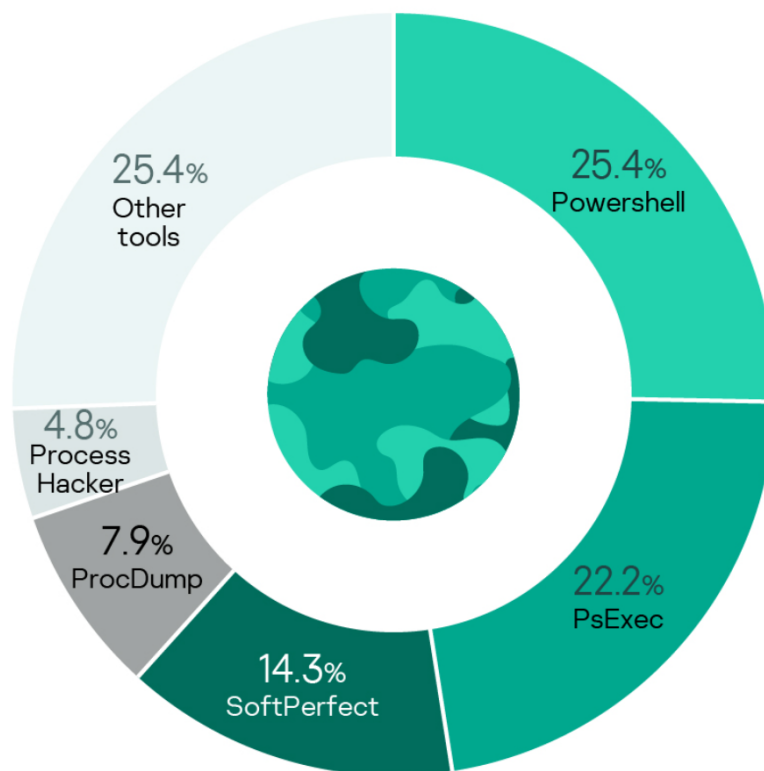


図 2.4 サイバー攻撃で使用された正規ツールの割合上位 [2]

2.3.5 PowerShell を用いたファイルレス攻撃のステップ

ファイルレス攻撃では、攻撃の全段階でファイルレスのテクニックを用いるとは限らない。攻撃者は、攻撃の数段階でファイルレスのテクニックを用いることが多い。ここで、攻撃者の攻撃目的を達成するまでに標的 PC で取る行動を段階化したものとして、サイバーキルチェーンがある。サイバーキルチェーンとは、LOCKHEED MARTIN 社が作成したサイバー攻撃を 7 つの段階に構造化したモデルである [8]。サイバーキルチェーンのモデルでは、サイバー攻撃が「偵察」、「武器化」、「デリバリ」、「エクスプロイト」、「インストール」、「Command & Control (以下、「C&C」という.)」、「目的の実行」の段階に分けられる。このうち、多くの攻撃者がファイルレス攻撃を用いるのは、「エクスプロイト」、「インストール」、「C&C」、「目的の実行」の段階である。ここで、サイバーキルチェーンにおける PowerShell を用いたファイルレス攻撃の例を、図 2.5 に示す。

まず、攻撃者は標的 PC に不正スクリプトが添付されたメールを送信する。標的 PC のユーザがその不正スクリプトを実行すると、PowerShell プロセスが起動する。以降、攻撃者は PowerShell を用いてファイルレス攻撃を実行する。攻撃者は起動した PowerShell で、悪意のあるペイロード

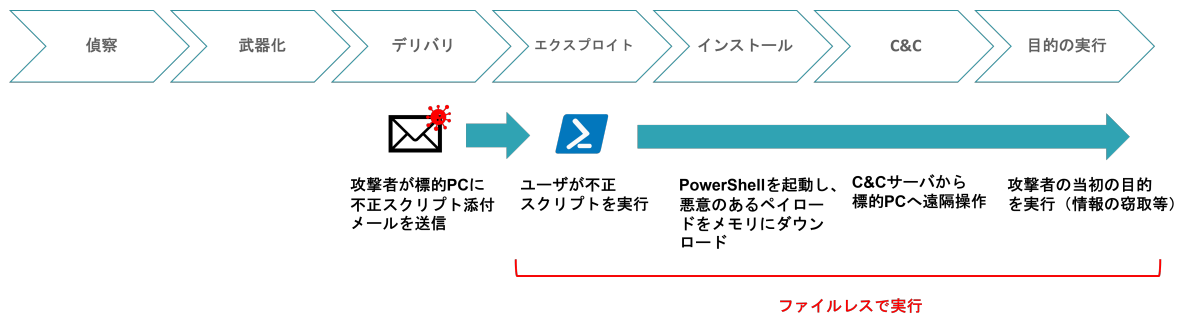


図 2.5 サイバーキルチェーンにおけるファイルレス攻撃の例

をメモリにダウンロードする。悪意のあるペイロードには、攻撃者が用意する C&C サーバから標的 PC へ遠隔操作可能な機能を保つ場合が多い。攻撃者は標的 PC と C&C サーバとの通信が確立できれば、C&C から標的 PC の PowerShell 上で任意のコマンドやスクリプトを実行できる。そして、攻撃者は攻撃目的を達成すれば、攻撃に関わるアーティファクトを削除しようと試みる。

第 3 章

準備

3.1 コンピュータセキュリティインシデント

3.1.1 コンピュータセキュリティインシデントの定義

NIST SP800-61 Rev.2 において、コンピュータセキュリティインシデントとは“組織が定めるセキュリティポリシーやコンピュータの利用規定に対する違反行為または差し迫った脅威，あるいは，標準的なセキュリティ活動に対する違反行為または差し迫った脅威”であると定義されている [9]。コンピュータセキュリティインシデントの具体例として，「サービス不能」，「悪意のコード」，「不正アクセス」や「不適切な使用」が挙げられる。

3.1.2 コンピュータセキュリティインシデント対応の必要性

NIST SP800-61 Rev.2 において，インシデント対応が必要な理由として“攻撃により個人データや業務データが頻繁に損なわれること”が挙げられている [9]。企業内機密情報の漏洩による企業に対する評価の低下や，システム停止によるサービス提供の停止が企業へ大きな損害を与えてしまう。そのため，コンピュータセキュリティインシデントが発生した場合，迅速かつ効率的に対応し，情報の損失や盗難，サービスの中断を最小限に抑えることが求められる。

3.1.3 コンピュータセキュリティインシデント対応の課題

2.2.1 項のとおり，ファイルレス攻撃はファイルを使用せずメインメモリ上で行われるため，標的マシンに攻撃に関わる痕跡が残りにくい。ファイルレス攻撃によるインシデントが発生した場合，インシデント対応の担当者は限られた情報から攻撃の原因を特定しなければならない。状況によっては，原因特定できる証跡が残っていない可能性もあり，インシデント対応が大幅に遅れてしまうことも考えられる。

ここでの証跡は，デジタル・フォレンジック研究会が発表した証拠保全ガイドライン [10] において，“コンピュータ・システムの仕様上，人や不正プログラムの操作により，ファイル／ネット

ワーク／内部等のさまざまな処理により必然的にディスクやメモリ上に残る痕跡のこと”と定義している。また英語圏における「Artifact（アーティファクト）」の概念や定義と近く、本論文では証拠を「アーティファクト」と呼ぶこととする。

3.2 デジタル・フォレンジック

3.2.1 概要

情報処理機器を対象とする不正侵入や情報漏えいのようなサイバー攻撃によるインシデントが発生した際、調査や原因究明を行う手法の一つとして「デジタル・フォレンジック」がある。デジタル・フォレンジック研究会によると、デジタル・フォレンジックとは“インシデントレスポンスや法的紛争・訴訟に対し、電磁的記録の証拠保全及び調査・分析を行うとともに、電磁的記録の改ざん・毀損等についての分析・情報収集等を行う一連の科学的調査手法・技術” [11] と定義している。NIST SP 800-86 によると、デジタル・フォレンジックの実施プロセスを「収集」、「検査」、「分析」、「報告」の4つに分類できるとしている [12]。デジタル・フォレンジック研究会によると、“この4つのプロセスの中で基本となるのは電磁的証拠の保全（Digital Evidence Preservation）の手続き” [10] だとしている。また、インシデントに関わるデジタル機器に残されたデータの中から、電磁的証拠となり得るものを、確実に“そのまま (As-is) で、収集 (Collection)・取得 (Acquisition)、保全 (Preservation) しておくことはデジタル・フォレンジックの運用者にとって最も重要なこと” [10] としている。

3.2.2 デジタル・フォレンジックの分類

コンピュータ・フォレンジック

コンピュータ・フォレンジックとは、“コンピュータやサーバ上のハードディスク等のデータを収集・解析することにより不正の証拠などを確保するための手順や技術のこと”である [11]。デジタル・フォレンジック研究会によると、コンピュータ・フォレンジックを「狭義のデジタル・フォレンジック」と呼んでいる [11]。ここで、本論文における「デジタル・フォレンジック」は主にコンピュータ・フォレンジックを指すこととする。一般的に、デジタル・フォレンジックで調査の対象となるのはコンピュータ内の補助記憶装置である。これは、補助記憶装置がコンピュータの電源供給を絶ってもデータが消失しない不揮発性であり、証拠となるデータの保存が容易なためである。

メモリ・フォレンジック

メモリ・フォレンジックとは、“メインメモリ（主記憶装置）上のデータのダンプなどを行った後、解析する手順や技術”である [11]。2.2 節で述べたように、サイバー攻撃の巧妙化によりコンピュータ内の補助記憶装置にアーティファクトを残さないようなファイルレス攻撃が増加している。ファイルレス攻撃によるインシデントが発生した際には、主に実施される補助記憶装置の解析では対応できない可能性があるため、メモリ・フォレンジックによる解析が有効である。一方で、

メインメモリは揮発性が高く、頻繁にメモリダンプを取得すると時間がかかりコンピュータの通常処理に支障を与えかねない。そのため、メモリ・フォレンジックにおいて、メインメモリ上のデータの保全の重要性がさらに高まってきている。

ライブ・フォレンジック

ライブ・フォレンジックとは、“起動中のコンピュータでリアルタイムに情報収集・解析を行うこと”である [11]。不揮発性の補助記憶装置の他に、メモリ・フォレンジックのような揮発性の高いメインメモリに対し情報収集や解析を行うときに有効である。一方で、ライブ・フォレンジックにおいては以下のような「システムに与える影響」、「再現可能性」、「情報収集システムの導入」の制約が挙げられる。

- システムに与える影響の制約として、起動中のコンピュータにおいて証拠取得行為自体が、証拠を改変する危険性を伴う。
- 再現可能性の制約として、ライブ・フォレンジックの性質上、情報収集や解析の結果の再現性を保証することが困難である。
- 情報収集システムの導入の制約として、ライブ・フォレンジックを実施するためには、事前に解析対象となり得るコンピュータに対して情報収集するシステムを導入する必要がある、ライブ・フォレンジックを実施できるのは一部のコンピュータに限られる。

ファスト・フォレンジック

ファスト・フォレンジックとは、“インシデントレスポンスや応急な捜査の対応のために証拠性確保の完全性をやや犠牲にしても、早急に結果を求めようとするデジタル・フォレンジックの手順や技術”である [11]。3.1 節で述べたように、インシデント対応が遅れるほど企業内機密情報の漏洩による企業に対する評価の低下や、システム停止によるサービス停止による企業の損害が増大してしまう。さらに、コンピュータ内の補助記憶装置の大容量化に伴い、インシデント発生時に証拠収集や解析にかかる時間が増大している。そのため、解析対象となるすべてのデータを収集・解析するのではなく、インシデントへの初動対応に必要なデータに絞って調査を進めることが求められる。

アンチ・フォレンジック

アンチ・フォレンジックとは、“デジタル・フォレンジックの証拠となる痕跡を残さないように、隠したり、改ざんしたり、消去したりする方法”である [11]。アンチ・フォレンジックの手法の一例として「ログシステムの難読化」や「ログの改ざん・削除」がある。従来のデジタル・フォレンジックが、主に補助記憶装置を対象とする点を踏まえると、ファイルレス攻撃もアンチ・フォレンジックの1つとして考えられる。攻撃者がアンチ・フォレンジック技術を用いることで、フォレンジック調査員によるインシデント調査の進行に支障が生じる。状況によっては、調査に必要なアーティファクトが残っていないという可能性や、調査で収集したアーティファクトが改ざんされてい

る可能性もある。そのため、デジタル・フォレンジックで得られたアーティファクトに対しても、常にアンチ・フォレンジックの可能性を疑うようにする必要がある。

3.2.3 ファイルレス攻撃に対するデジタル・フォレンジック

デジタル・フォレンジックにおける主な調査対象となるのは補助記憶装置である。しかし、ファイルレス攻撃の増加により、インシデント調査において、攻撃に関するアーティファクトが補助記憶装置に残っていない状況も考えられる。そのため、ファイルレス攻撃への主な対応として、メモリ・フォレンジックのような主記憶装置を対象とした調査もしくは、ログ解析が挙げられる。3.2.2 項の通り、メモリ・フォレンジックのようなライブ・フォレンジックにはいくつかの制約があるため、発生したすべてのインシデントにおいてメモリ・フォレンジックを進めることは難しい。2.2 節で述べたように、多くのファイルレス攻撃は PowerShell のような標的マシン内の正規ツールを悪用する。一般的に、多くの正規ツールには、使用者の操作情報や実行処理に関わる情報をログとして記録する機能がある。そこで、ファイルレス攻撃に対するデジタル・フォレンジックの調査手法として、それらのログを用いた解析が行われる。そのため、ログはファイルレス攻撃に対する調査で重要なアーティファクトとなる。

3.3 コンピュータセキュリティログ

NIST SP 800-92 によると、ログとは“組織のシステムおよびネットワーク内で発生するイベント（事象）の記録”と定義されている [13]。コンピュータセキュリティログ（以下、「ログ」とする）はウイルス対策ソフトウェア、ファイアウォール、オペレーティングシステムおよびアプリケーションをはじめとする数多くの情報源によって生成される。インシデントが発生した際、ログはデジタル・フォレンジックの解析を進める重要なアーティファクトとなる。

3.3.1 ログ管理

ログ管理とは、“コンピュータセキュリティログデータの生成、通信、格納、分析、廃棄するプロセスのこと” [13] である。ログ管理にはログの CIA（機密性（Confidentiality）、完全性（Integrity）、可用性（Availability））を守ることも含まれている [13]。ログの CIA を守る課題として、ログを確実に保管することに加え、セキュリティやシステムおよびネットワークの管理者によって定期的の実効性のあるログデータの分析を行うことが挙げられる。

3.3.2 ログの保護

生成されたログは保存時に適切なセキュリティ保護が施されない場合、作為の有無にかかわらず改変や破損を受ける可能性がある。多くの攻撃者は、アンチ・フォレンジックの技術を攻撃プロセスの中に組み込み、攻撃に関わるログの改変や削除を試みる。ログの改変や削除を組み込んだ攻

撃ツールの例として、「rootkit（ルートキット）」がある。rootkit は、標的のコンピュータにおいて継続的な攻撃を仕掛けるためのインストールツールがパッケージ化されたものである。多くの rootkit には、攻撃に関わるログを削除するような隠蔽機能が備わっている。隠蔽機能のようなアンチ・フォレンジックの手法に対しても、ログを確実に保護することが必要である。また、保存環境にもよるが、ログを保存できる量には上限がある。上限に達すると、新しいデータによる古いデータの上書きあるいは、ログの記録が停止し、すべてのログを確実に保存できない。

3.3.3 Windows イベントログ

Windows イベントログとは、Windows システムにおいてアプリケーションやシステムサービスに関する重要な情報がイベントとして記録されるログである。PowerShell のようなコマンドラインツールもしくは、Windows に標準搭載されているアプリケーションの Event Viewer（イベントビューア）において、Windows イベントログを確認できる。Windows イベントログシステムでは取得するログを細かく設定できるが、デフォルトで取得できるログは限られるため、追加でポリシーの設定が必要となる。

3.4 PowerShell ログシステム

Microsoft 社のドキュメントによると、PowerShell ログシステムについて“PowerShell は、エンジン、プロバイダ、コマンドレットからの内部操作を Windows イベントログに記録する” [14] と説明している。

3.4.1 PowerShell ログシステムの仕様

PowerShell ログシステムの仕組みを図 3.1 に示す。PowerShell ログシステムの設定は、Windows のレジストリで管理されており、PowerShell の管理者がレジストリを直接操作もしくは、グループポリシー（GPO）を操作して設定できる。PowerShell の管理者がデフォルト設定で取得できるログは、PowerShell の起動に関するログのみである。そのため、詳細な PowerShell ログを取得するためには、追加で設定が必要である。

PowerShell ログシステムの設定は、レジストリキー“`\HKEY_LOCAL_MACHINE\SOFTWARE\Policies\Microsoft\Windows\PowerShell\`”以下にある。PowerShell ログシステムは PowerShell の起動時に上記のレジストリキーの値を System.Management.Automation.Utils の `cachedGroupPolicySettings` フィールド（PowerShell 7 では、`s_cachedPoliciesFromRegistry` フィールド）にキャッシュする。ここで重要なことは、PowerShell の実行時に PowerShell ログシステムが上記のレジストリキーの値を毎度参照せず、PowerShell 起動時にキャッシュした値を参照するということである。これは、PowerShell の実行パフォーマンスをあげるためである。また、PowerShell ログシステムで記録されたログは、Windows イベントログの一部としてローカルマシン内に保存される。

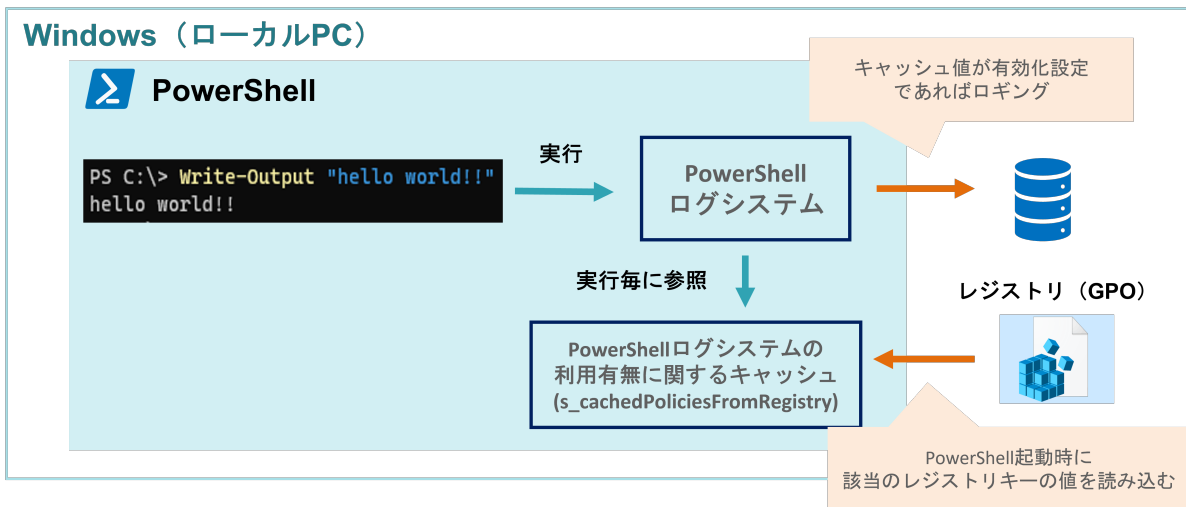


図 3.1 PowerShell ログシステムの仕組み

3.4.2 PowerShell ログの種類

PowerShell ログシステムには「Module Logging」、「Script Block Logging」、「Transcription Logging」の 3 つの機能がある。

Module Logging

Module Logging は、PowerShell においてユーザが実行したコマンドの呼び出しやスクリプトの一部を記録する機能である。また、コマンドに設定されている引数の情報も詳細に記録される。Windows イベントログでは、Module Logging で記録されたログ（以下、「モジュールログ」とする）が Event ID: 4103 として記録される。図 3.2 に Event Viewer においてモジュールログの例を示す。プログラムの動作を変えずにコードを読みにくくする手法である難読化を適用されたコマンドにおいても、難読化解除された生のコマンド情報が記録される。また、実行されるコマンドが変数およびエイリアスを介する場合でもそれらの参照元のコマンドが記録される。モジュールログは、主にユーザが実行したスクリプトで使用されているコマンドを抽出するときに利用される。ただし、Module Logging は実行したスクリプト全内容は記録しない。Module Logging は、Windows PowerShell 3.0 以降において使用可能で、デフォルト設定では無効化されている。そのため、グループポリシーの設定により有効化する必要がある。

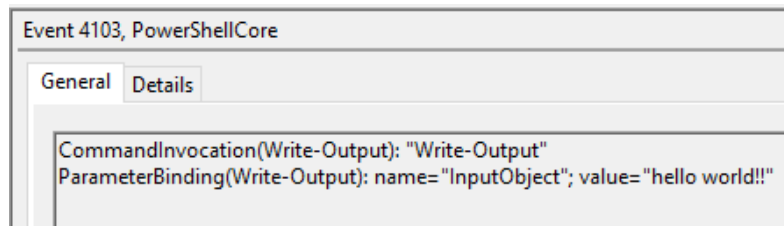


図 3.2 モジュールログの例

Script Block Logging

PowerShell では、プログラムが実行される処理を「ステートメント」と表す。Script Blocks（スクリプトブロック）とは、“1 つのユニットとして使用できるステートメントまたは式のコレクション”である [15]。Script Block Logging は、ユーザが PowerShell で実行したすべてのコマンドおよびスクリプトの内容を「Script Block」単位で記録する機能である。Windows イベントログでは、Script Block Logging で記録されたログ（以下、「スクリプトブロックログ」とする）が Event ID: 4104 として記録される。図 3.3 に Event Viewer においてスクリプトブロックログ例を示す。PowerShell でのユーザの操作履歴を確認するためには、スクリプトブロックログを参照することが有効である。そのため、一般的にデジタル・フォレンジックの調査員は PowerShell ログシステムで記録されるログの中で、一番初めにスクリプトブロックログを確認する。Script Block Logging は、Module Logging のように、難読化されたコマンドおよびスクリプトでも大部分は難読化解除されたものが記録される。ただし、変数やエイリアスおよび一部の難読化手法に対しては完全に難読化解除されずに記録される。このログ機能は、Windows PowerShell 5.0 以降において使用可能で、デフォルト設定では無効化されている。そのため、Module Logging と同様、グループポリシーの設定により有効化する必要がある。

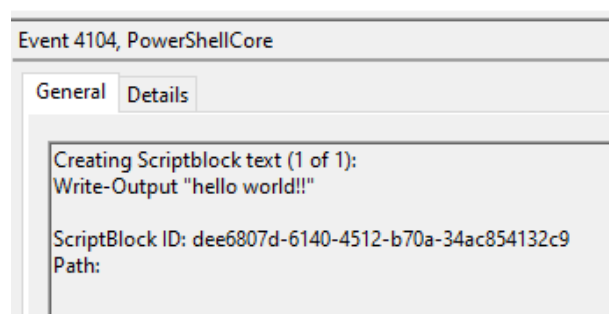


図 3.3 スクリプトブロックログ例

Transcription Logging

Transcription Logging は、“ユーザが入力したすべてのコマンドと、コンソールに表示されたすべての出力” [16] を記録する機能である。図 3.3 に PowerShell コンソールにおける Transcription Logging で記録されたログ(以下、「トランスクリプションログ」とする)の例を示す。Transcription Logging は、Module Logging と Script Block Logging と同じくグループポリシーで設定できる。さらに、PowerShell コンソール上で Transcription Logging を有効化および無効化できる標準コマンドがあり、Start-Transcript コマンドレットで有効化、Stop-Transcript コマンドレットで無効化できる。そのため Transcription Logging は、攻撃者によってコンソール上から容易に無効化できることからデジタル・フォレンジックの調査で確実に利用できるという保証はない。

```
PS C:\Users\cysec> Start-Transcript -Path .\tslog.txt
Transcript started, output file is .\tslog.txt
PS C:\Users\cysec> Write-Output "hello world!!"
hello world!!
PS C:\Users\cysec> Get-Content .\tslog.txt
*****
PowerShell transcript start
Start time: 20230106190101
Username: VM-WINDOWS10-21\cysec
RunAs User: VM-WINDOWS10-21\cysec
Configuration Name:
Machine: VM-WINDOWS10-21 (Microsoft Windows NT 10.0.19044.0)
Host Application: C:\Program Files\PowerShell\7\pwsh.dll
Process ID: 10704
PSVersion: 7.2.8
PSEdition: Core
GitCommitId: 7.2.8
OS: Microsoft Windows 10.0.19044
Platform: Win32NT
PSCompatibleVersions: 1.0, 2.0, 3.0, 4.0, 5.0, 5.1.10032.0, 6.
PSRemotingProtocolVersion: 2.3
SerializationVersion: 1.1.0.1
WSManStackVersion: 3.0
*****
Transcript started, output file is .\tslog.txt
PS C:\Users\cysec> Write-Output "hello world!!"
hello world!!
PS C:\Users\cysec> Stop-Transcript
Transcript stopped, output file is C:\Users\cysec\tslog.txt
PS C:\Users\cysec> |
```

図 3.4 トランスクリプションログの例

3.5 PowerShell ログシステムとアンチ・フォレンジック

3.2.3 項で述べたように、PowerShell を用いたファイルレス攻撃に対するデジタル・フォレンジックの調査手法として、主にログ解析が用いられる。一方で、多くの攻撃者は、攻撃プロセスの中にアンチ・フォレンジック手法を搭載することで、攻撃に関わるログを残さないようにする。そのため、インシデント調査において、デジタル・フォレンジック調査員が収集・解析可能なログが極端に少なくなっている。

PowerShell ログシステムに対するアンチ・フォレンジック手法として、「PowerShell ログシステムの無効化」、「PowerShell ログの改ざん・削除」、「PowerShell スクリプトの難読化」があげられる。

3.5.1 PowerShell ログシステムの無効化

PowerShell ログシステムを有効化・無効化するためには、管理者権限によるレジストリもしくはグループポリシーの変更が必要である。つまり攻撃者が、PowerShell ログシステムを無効化するためには管理者権限が必要となる。ただし、PowerShell ログシステムの Transcription Logging については、攻撃者がコンソール上から管理者権限不要で容易に有効化・無効化できる。

Ryan Cobb が発表した PowerShell ログシステムの無効化手法に “PowerShell ScriptBlock Logging Bypass” がある [17]。図 3.5 に示すように、この手法では、PowerShell 起動時に該当レジストリキーから読み込まれた PowerShell ログシステムに関わるキャッシュの値を書き換えることで、Script Block Logging を無効化する。Module Logging についても、Script Block Logging と同様に、PowerShell ログシステムに関わるキャッシュの値を書き換えることで、Module Logging を無効化する。本論文では、この手法を Script Block (Module) Logging Bypass と呼ぶこととする。この手法の強力な点として、攻撃者が PowerShell コンソール上から管理者権限不要でキャッシュ値書き換えて、無効化できることである。さらに、この手法はすでに起動された PowerShell 内のキャッシュ値のみを書き換えるため、該当のレジストリキーの値は変更されない。これにより、この手法は Microsoft Defender をはじめとするアンチ・ウイルスソフトウェアに備わるレジストリ変更の監視機能に対しても有効である。

この手法を用いた攻撃事例として、2017 年 7 月にアメリカ学会に対して PowerShell Empire を利用した標的型攻撃が挙げられる [18]。PowerShell Empire とは、レッドチームやペネトレーション実施者を支援するために開発されたポストエクスプロイトフレームワークである。PowerShell Empire は、パスワードダンプやネットワーク探索といったシステムへの侵入後に利用するモジュールを数多く備えている。さらに、Script Block (Module) Logging Bypass の手法を用いた PowerShell ログシステムの無効化機能も実装されている。LAC 社が発表したレポートによると、2016 年頃からこの事例のように公開されているペネトレーションツールを流用または応用した攻撃事例が散見されるようになったと報告している。つまり、今後も PowerShell ログシステムの無

効化を組み込んだ攻撃事例が増加すると考えられる。

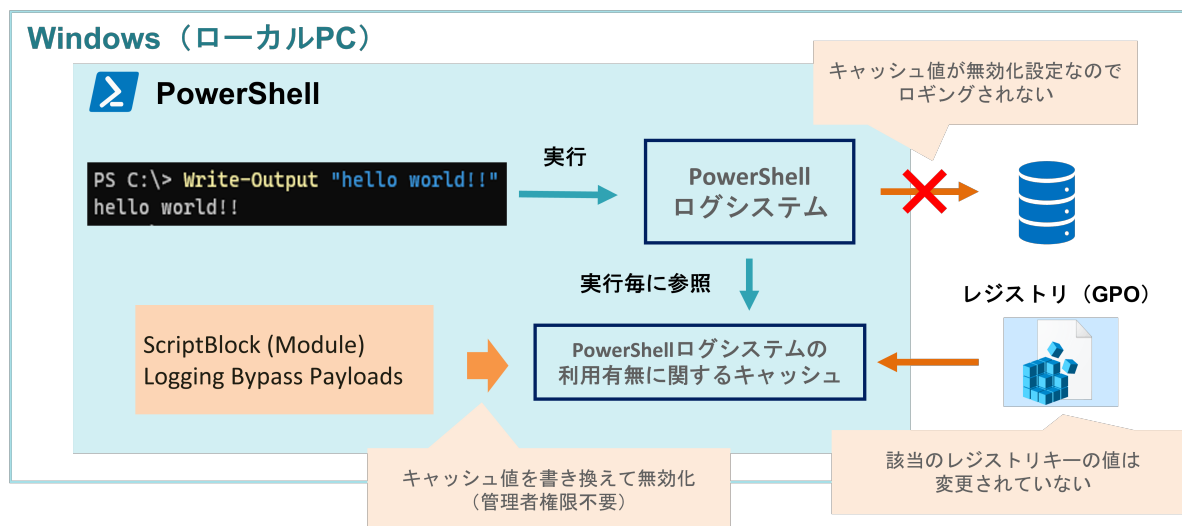


図 3.5 Script Block (Module) Logging Bypass

3.5.2 PowerShell ログの改ざん・削除

Windows イベントログの 1 つとして管理されている PowerShell ログは通常、PowerShell を起動しているローカルマシン内に保存されている。そのため、攻撃者によってローカル内に保存されたログを特定されると、改ざん・削除の影響を受ける可能性がある。ただし、ログの改ざん・削除には管理者権限が必要である。

PowerShell ログの改ざん・削除の手法を用いた攻撃事例として、2021 年 5 月に Sophos 社のアナリストが発見した「Epsilon Red」と呼ばれるランサムウェアを用いた攻撃が挙げられる。同社が発表したレポートによると [19]、この攻撃では攻撃初期段階で PowerShell スクリプトを多用しており、攻撃プロセスの中で PowerShell ログを含めた Windows イベントログを削除する機能を実行する。

3.5.3 PowerShell スクリプトの難読化

ソフトウェアにおける難読化とは、プログラムの動作を変えずにコードの可読性を下げて、解析を妨害する行為である。攻撃者は、攻撃プログラムに難読化手法を適用することで、フォレンジック調査員の解析の妨害に加えて、アンチ・ウイルスソフトウェアによる検知を回避できる。表 3.1 に示すように、分割やエンコードといった文字列の操作をはじめとする、PowerShell スクリプトに対する難読化手法は数多く存在する。攻撃者は、単一の難読化手法を用いるのではなく複数の難読化手法を組み合わせることで、容易に難読化解除されないようにする。

表 3.1 PowerShell スクリプトの難読化手法の例

手法	入力例 *
分割	Write-Output("hello" + "world!!")
順番の入れ替え	Write-Output("{1}{0}" -f 'world!!', 'hello ')
置換	Write-Output("hexxo worxd!").Replace('x','l')
型のキャスト	Write-Output("hello " + [char]119 + "orld!!")
エンコード	.([stRing]\$vErbOsepreFEREnCe)[...(略
バッククオート	Write-Output `h`E`l`l`o` `W`o`R`l`d`!`
スペース	Write-Output("hello world!! ")
変数	\$print="Write-Output"; \$hw="hello world!!"; &\$print(\$hw)
省略名使用	&(GCM *te-Ou*)"hello world!!
大小文字混合	wRiTEmOuTPUT "hELlo WOrLD!!"

* 「Write-Output "hello world!!"」スクリプト（ここでは、「平文スクリプト」と呼ぶ）に対し、実行結果を変えずに上記の各難読化手法を適用する。

3.5.4 難読化と PowerShell ログシステム

PowerShell では、標準で用意されているコマンドレットに加えて、クラス・メソッドを自ら定義もしくは.NET クラス・ライブラリの読み込みで利用できる。表 3.1 のような手法で難読化された PowerShell のコマンドレット・スクリプトおよびクラス・メソッドに対する標準ログシステムの難読化解除の可否を、表 3.2 に示す。

表 3.2 標準 PowerShell ログシステムの難読化解除の可否

難読化対象	Script Block Logging	Module Logging
コマンドレット	△ ^{*1}	○
クラス・メソッド	△	— ^{*2}
スクリプト（ブロック）全体	△	—

^{*1} 一部の難読化手法に対しては解除できる。（表 3.1 の手法のうち、「エンコード」のみ解除可能）

^{*2} 難読化関係なく、ログとして記録されない。

難読化されたコマンドレットの実行に対し、標準の PowerShell ログシステムの Script Block Logging では完全に解除できないが、Module Logging では解除できる。クラス・メソッドおよ

びスクリプト（ブロック）全体は，Script Block Logging で完全に解除できないことに加えて，Module Logging ではログ自体が記録されない．このように，標準ログシステムでは難読化されたクラス・メソッドの実行に対し，難読化解除されたログを取得できない．

第 4 章

関連研究

4.1 ログの保全に関する関連研究

4.1.1 ヒステリシス署名

芦野ら [20] は、セキュリティデバイスとヒステリシス署名を用いてログファイルを保全する手法を提案している。ヒステリシス署名とは、対象データを署名する際に過去に署名したデータ（署名履歴）を含めて、署名処理を行うものである。これにより、署名されたログファイル間に連鎖構造が築かれるため、攻撃者による一部のログファイルの改ざんを検知できる。本評価において、署名対象としたログはフリーソフトウェアの logol2 での操作ログである。logol2 は、アクティブウインドウのタイトルに加え、ユーザによるキーボード入力情報をログとして記録する。本システムでは、logol2 によるログの生成の監視を行い、生成されたログに対しヒステリシス署名を行う。ヒステリシス署名は署名の連鎖をたどることでファイルを改変される可能性があるため、芦野らの手法では、セキュリティデバイス内の耐タンパ領域にヒステリシス署名時の署名を保管している。この手法の問題点として、このヒステリシス署名機能がログを取得する機能と同じユーザ空間に存在することである。攻撃者がこのヒステリシス署名機能を改ざんおよび停止できる可能性があるため、ログファイルの完全性が保証されない。

4.1.2 逃げログ

高田ら [21] は、ログファイルのバックアップを複数（任意数）作成し、ローカル内のファイルシステムに保存することでログファイルの改ざんを困難にする手法を提案している。本手法では、定期的に原本のログファイルとバックアップのログファイルを比較することで、改ざんの有無を検知し、バックアップから改ざん内容を修正できる。ログファイルのバックアップは任意の数で作成され、書き込み可能な任意のディレクトリに隠蔽される。攻撃者によるログの改ざん・削除に対し、原本のログファイルおよびバックアップのログファイルのいずれか 1 つでも残存していれば、ログの復元は可能である。本研究では、UNIX 系 OS で動作する種々のアプリケーションのログの保全を想定している。本提案システムは汎用性を意識し、標準規格によるライブラリを使用した C++

のクラス・ライブラリとして実装されている。そのため、種々のアプリケーションに組み込むことができ、ログの保護処理を容易に実現できる。本研究の問題点として、ログファイルの安全性が隠蔽箇所数に依存することである。バックアップの隠蔽場所は、書き込み可能なディレクトリからランダムに選択される。ログを生成するアプリケーションが一般ユーザ権限の場合、書き込み可能なディレクトリが限定され、バックアップの安全性を低下させる可能性がある。そのため、攻撃者により攻撃対象のファイルシステムにある原本のログファイルおよびバックアップのログファイルを発見されやすくなり、改ざんや削除されると復元が困難になる。また、本提案システムは処理を継続して行う必要があるため、コマンド実行のような、短時間で処理されるアプリケーションのログ出力に適用することが困難である。また、原本のログファイルに加えてそのバックアップファイルも保存するため、ローカルのファイルシステムへの負荷も増大する。

4.2 関連研究と PowerShell ログシステム

4.2.1 ログシステムの完全性

4.1.1 項の関連研究の手法は、「ログシステムが正常に機能している」という前提がある。図 4.1 に示すように、攻撃者によりログシステムが無効化されると、ログの記録ができない。つまり、攻撃者によりログシステムが無効化された場合、ファイルに書き出されたログに対し保全を行う関連研究の手法では、ログを確実に保全できない。ログシステムを無効化する方法として、Windows のグループポリシー設定のような管理者が設定する方法と、アプリケーションごとに搭載するログシステム固有の仕様（脆弱性）を利用して無効化する方法がある。前者において、攻撃者は攻撃プロセスの中で標的 PC の管理者権限を奪取できた場合に、ログシステムを無効化できる。攻撃者は管理者権限によるグループポリシーの設定を行うことで、ほとんどの Windows 標準搭載アプリケーションのログシステムを無効化できる。後者の例として、主に PowerShell ログシステムが挙げられる。PowerShell ログシステムにおいては、攻撃者が 3.5.1 項で述べたような PowerShell ログシステムの無効化手法を用いることで無効化されてしまう。

このように、ログシステム無効化が可能な環境では、ログの保全プロセスにおいて、ログシステムがボトルネックとなる可能性がある。ログシステムが正常に機能しログを記録しなければ、関連研究の手法によるログの保全を実施できない。そのため、ログの保全において「ログシステムが正常に機能している」というログシステムの完全性を保つことが重要である、

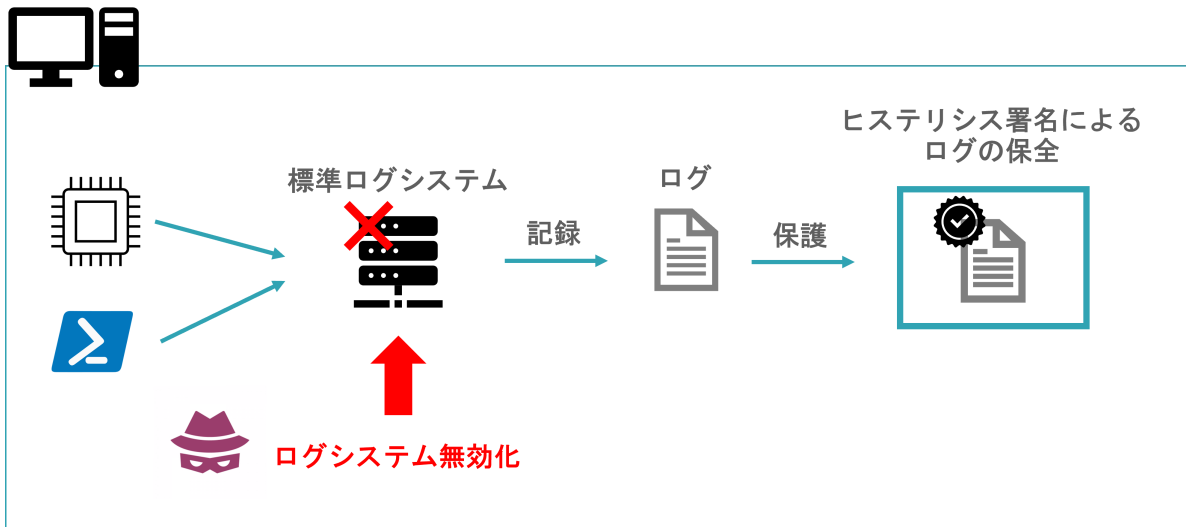


図 4.1 ヒステリシス署名によるログの保全とログシステムの無効化

4.2.2 ログの保護

4.1.2 項の関連研究の手法を用いて実装されたシステム（以下、「逃げログシステム」とする）は、種々のアプリケーションに組み込まれる。そのため、4.2.1 項で述べたように、逃げログシステムは標準ログシステムよりも攻撃者によるログシステムの無効化の影響を受けにくい。逃げログシステムにおいて、ログの原本およびそのバックアップはローカル内のファイルシステムに隠蔽される。4.1.2 項で述べたように、逃げログシステムを組み込むアプリケーションが一般ユーザ権限で実行される場合、書き込み可能なディレクトリ数が減少し、バックアップを隠蔽するディレクトリが限定される。その場合、攻撃者はログファイルの原本およびバックアップファイルを発見し、改ざんや削除が容易となる。また、本手法はコマンド実行のような短時間で処理されるアプリケーションに適用できない。したがって、PowerShell に逃げログシステムを組み込むことは困難である。

4.1.1 項の関連研究の手法において、ヒステリシス署名を施したログに対する攻撃者の改ざん・削除を検知できる。一方で、本手法では攻撃者によるログの改ざん・削除を防止できない。3.2.3 項で述べたように、ファイルレス攻撃に対するフォレンジック調査として主にログ解析が行われる。ログはファイルレス攻撃に対する調査で重要なアーティファクトであり、ログを正確に保全する必要がある。そのため、攻撃者によるログの改ざん・削除の検知だけでなく、防止することが必要である。

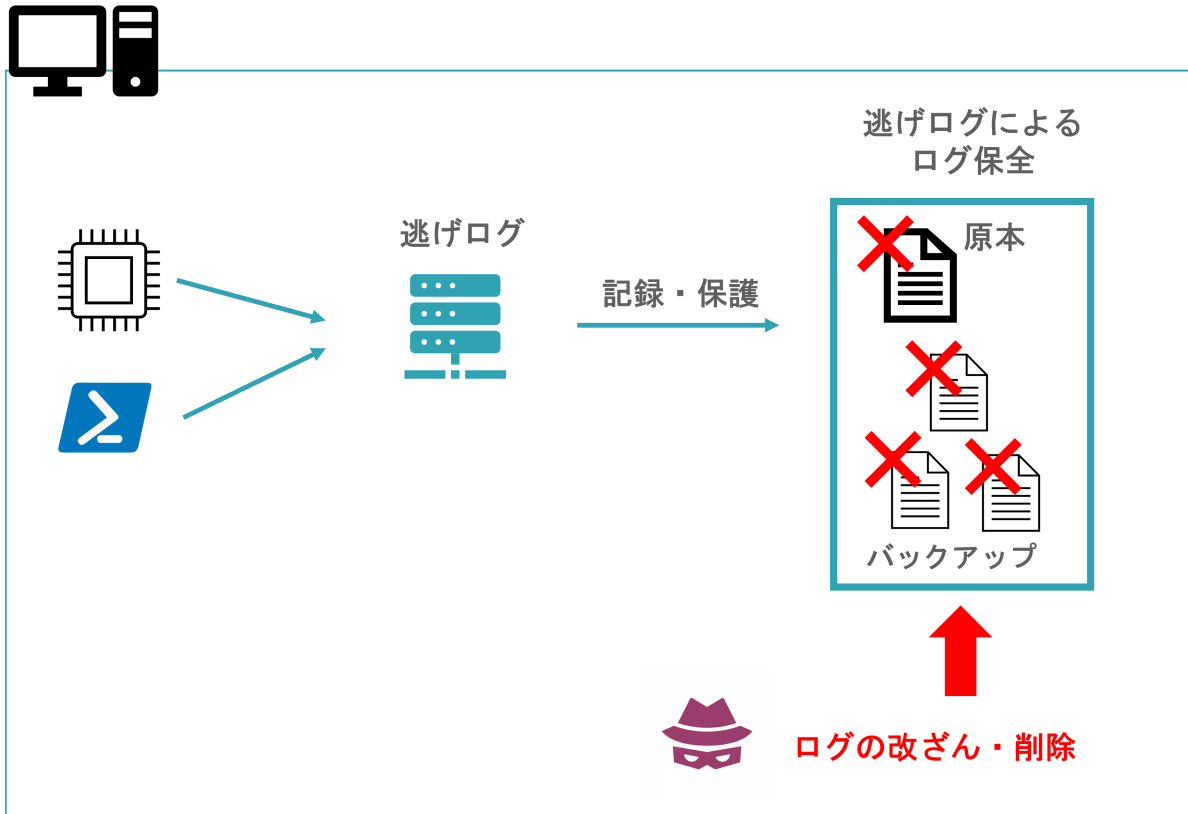


図 4.2 逃げログによるログの保全とログの改ざん・削除

4.2.3 関連研究の課題

多くの攻撃者は、ログシステムおよびログシステムで記録されたログに対し、アンチ・フォレンジックを実行する。攻撃者によってアンチ・フォレンジックの1つであるログシステムの無効化手法が成功した場合、ログが記録されない。この場合、4.1.1 項の芦野らが提案したようなログシステムで記録されたログを保全する手法を用いても、ログを保全できない。一方で、ログシステムは正常に機能しているとしても、記録されたログを攻撃者によって改ざん・削除された場合、ログを保全できない。4.1.2 項の高田らが提案した手法では、ログシステムをアプリケーション内部に組み込むため、ログシステム無効化に対し耐性があるが、記録されたログの保全は十分に保証されているとは言えない。このように関連研究では、攻撃者がログシステムの無効化およびログの改ざん・削除の機能を攻撃に組み込んだ場合、ログを確実に保全できない。そこで、本研究では攻撃者がログ保全に対するアンチ・フォレンジックを組み込んだとしても、PowerShell の操作に関わるログを確実に保全する。

第 5 章

提案手法

5.1 概要

PowerShell を用いたファイルレス攻撃によるインシデントが発生した場合、多くのフォレンジック調査員はログ解析を実施する。PowerShell ログシステムに対するアンチ・フォレンジック手法を組み込んだ攻撃においても、フォレンジック調査に利用できるログを確実に保全することが必要である。本研究では、アンチ・フォレンジックとして、PowerShell ログシステムの無効化手法および、PowerShell ログの改ざん・削除を想定している。現状、3.4.1 項で述べたような標準の PowerShell ログシステムはその仕様上、3.5 節のアンチ・フォレンジック手法を組み込んだ攻撃に対して、ログを保全できない。また、4.2.3 節で述べたように、関連研究の提案手法ではログシステムの無効化およびログの改ざん・削除に対し、ログを確実に保全できない。そのため、PowerShell ログシステムに対し、関連研究の提案手法を適用してもログを確実に保全できない。

そこで本研究では、図 5.1 に示すように、PowerShell 内部に独自ログシステムを構築し、直接ローカル外ログサーバ（以下、「ログサーバ」とする）にログを保全する手法について提案する。ここで、ログサーバも含め、本提案手法に従って構築されるシステム全体のことを「独自保全システム」と呼ぶこととする。また、独自ログシステムを内部に組み込んだ PowerShell のことを「カスタム PowerShell」と呼ぶこととする。構築する独自ログシステムは、ユーザによるログの有効化・無効化設定機能を持たない。そのため、外部から独自ログシステムに干渉できず、強制的にログを記録できる。これにより、PowerShell ログシステムの無効化手法を組み込んだ攻撃に対しても、ログを確実に保全できる。さらに、独自ログシステムからログサーバにログを転送するため、ローカル内にログが残らず攻撃者による PowerShell ログの改ざん・削除手法の影響も受けないと考えられる。

本研究で提案する独自ログシステムは PowerShell 内部に組み込むため、PowerShell 内部処理レベルでロギングが可能である。ここでの PowerShell 内部処理の一例として、(.NET) クラス・メソッド、レジストリへのアクセス、変数処理およびエンコード・デコード処理が挙げられる。実行されるコマンドおよびスクリプトが難読化されている場合でも、内部処理では難読化解除される。そこで本研究では、独自ログシステムから PowerShell 内部処理をフックすることで、難読化され

た PowerShell スクリプトの解除を目指す。

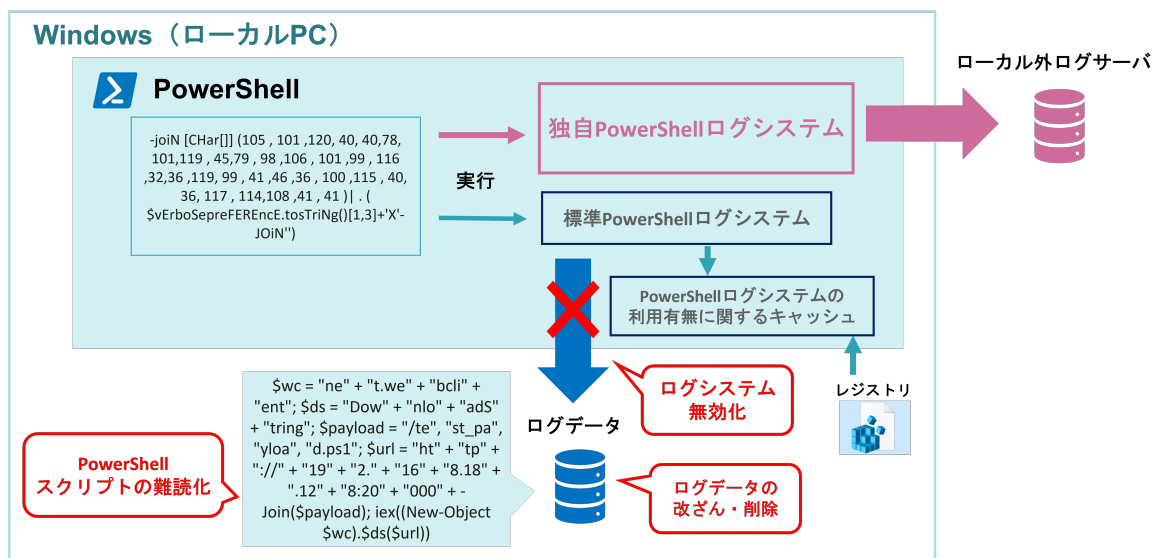


図 5.1 提案手法の概要

5.2 独自保全システムの全体像

独自ログシステムの動作は以下のような流れで行われる。また、独自ログシステムの動作全体像を図 5.2 に示す。

1. PowerShell プロセスの起動後に、独自ログシステムがログサーバに署名用の公開鍵を送信する。
2. ログサーバは受け取った公開鍵で暗号化した共通鍵を独自ログシステムに送信する。
3. 独自ログシステムはログデータ（送信ログ）をローカルストレージに保存およびログサーバに送信する。
4. ログサーバは受け取った送信ログに付加されている署名とハッシュ値を検証する。（ただし、最初の送信ログの場合はハッシュ値検証を行わない。）
5. ログサーバは検証結果を確認応答として独自ログシステムに送信する。
6. 独自ログシステムは受け取った確認応答の内容が検証成功であれば、ローカルストレージに一時保存されているログデータを削除する。検証失敗もしくは確認応答が送信されなければ、独自ログシステムがローカルストレージにあるログデータをログサーバに再送する。

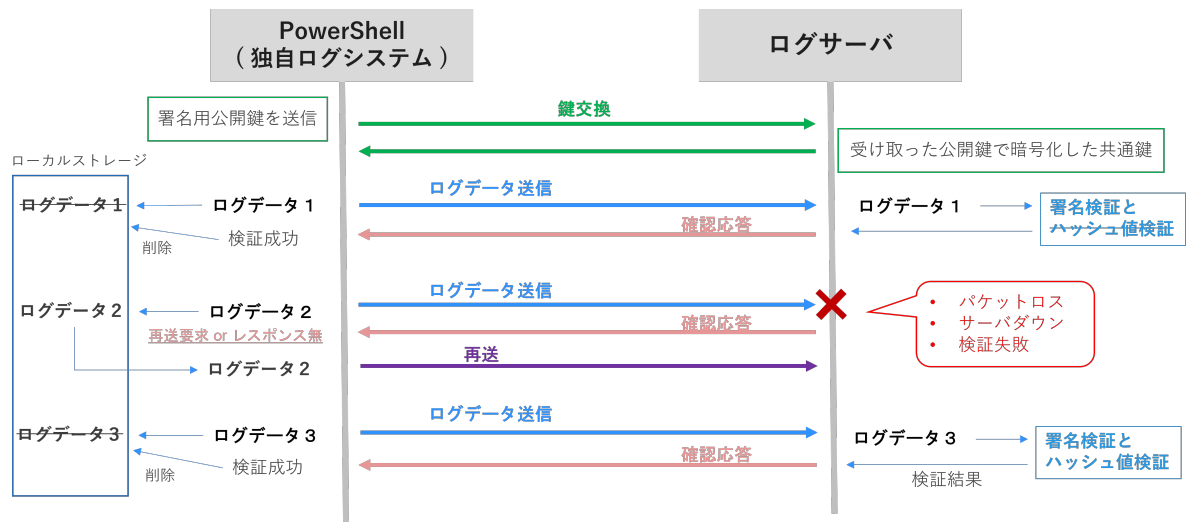


図 5.2 独自保全システムの動作の流れ

5.3 保全のための要件

ログを確実に保全するという点において、独自ログシステムは外部からの干渉を許さず、常時稼働できることが最も重要である。さらに、ログをログサーバに転送する点において、ログが独自ログシステムからログサーバに確実に転送・記録されることを保証する必要がある。そこで、独自ログシステムにおいて、ログを確実に保全するために以下 5 つの要件を満たす必要があると考える。

- 独自ログシステムは、常に稼働できること。
- ログサーバが受け取るログデータが独自ログシステムから送られたものか判別できること。
- ログ転送路におけるログの改ざんや欠損を検知できること。
- ログサーバに記録されるログの順序が、独自ログシステムで記録されたログの順序と同じであること。
- 独自ログシステム、ログサーバおよび通信路における障害発生に対応できること。

5.4 独自保全システムの実現

独自保全システムの提案にあたり、5.3 節で挙げた保全要件を満たすための独自保全システムの機能について検討する。

5.4.1 独自ログシステムの組み込み

本提案手法では、独自ログシステムを PowerShell 実行処理系内にビルドする形で組み込む。つまり、独自ログシステムは PowerShell 実行プロセス内で稼働することになる。そのため、攻撃者を含めたすべての PowerShell ユーザは、独自ログシステムの存在を認知できても、独自ログシステムに干渉することが困難である。一方で、PowerShell 本体の中に独自ログシステムを組み込むため、PowerShell 本体の処理に影響を与える可能性がある。そこで、PowerShell 本体の処理にできる限り影響を与えないようにする必要がある。

5.4.2 ログサーバへの保全

独自ログシステムで記録されたログは、ローカル外のログサーバに送信される。そこで安全で確実なログ保全を実現するためには、独自ログシステムとログサーバおよび通信路に対し、障害発生を想定する必要がある。そこで、本手法では独自ログシステムがログサーバに対し、安全にログを送信・保全できるように、「署名検証」、「直前ログデータのハッシュ値検証（以下、「ハッシュ値検証」と呼ぶ）」、「再送機能」を組み込んだ独自保全システムを検討する。

署名検証

ログサーバは、独自ログシステムから送信されたログのみを受け付ける必要がある。また、ログサーバは受け取ったログが第三者によって改ざんされていないことを確認する必要がある。そこで本手法では、ログサーバに送信するログに署名を付加する。独自ログシステムおよびログサーバにおける署名検証を、以下の流れで行う。

1. 図 5.2 の鍵交換フェーズにおいて、独自ログシステムは鍵ペア（秘密鍵および公開鍵）を生成し、ログサーバに公開鍵を送信する。
2. 独自ログシステムは、ハッシュ関数を用いてログのハッシュ値を算出する。
3. 独自ログシステムは、そのハッシュ値に対し、独自ログシステムが持つ秘密鍵で暗号化する。（暗号化したものが、署名となる。）
4. 独自ログシステムは、生成した署名をログに付加し、それをログサーバに送信する。
5. ログサーバは、受け取ったログの署名を、1. で独自ログシステムから受け取った公開鍵で復号する。
6. ログサーバは、独自ログシステムと同じハッシュ関数を用いて、ハッシュ値を算出する。
7. ログサーバは、5. で復号したハッシュ値と、6. で算出したハッシュ値を比較して一致すれば、署名検証成功とする。

ログサーバにおいて署名の検証に成功すれば、独自ログシステムからログサーバに送信されたログが「独自ログシステムから送信されたもの」および「改ざんされていない」ということを証明で

きる。

ハッシュ値検証

インシデント調査員によるログ解析の妨害を目的として、攻撃者はログサーバに対し、独自ログシステムで記録されたログ以外のデータを送信する可能性がある。本手法ではログデータの署名検証の機構を設けているため、ログサーバは独自ログシステム以外からのデータを受け付けない。一方で、攻撃者が独自ログシステムとログサーバの通信路に流れる正規のログを盗聴・複製し、そのままログサーバに再送する可能性が想定できる。その場合、ログサーバは、攻撃者によって複製されたログを正規の送信ログとして処理してしまう。また、本手法における独自ログシステムとログサーバの通信では、UDP を用いる（6.2.1 項で述べる）。そのため、ログサーバへの送信ログの到着順序が保証されず、記録するログの順序が変わる可能性がある。

そこで本手法では、図 5.3 のように、直前の送信ログデータのハッシュ値を次のログデータに付加し連鎖構造にすることで、非正規ログの差し込みを検知可能にする。ログサーバは、直前の送信ログデータのハッシュ値と次のログデータに付加されたハッシュ値を検証し、両者が一致すれば正規のログとして受け付ける。このように、ログデータ間の連鎖構造を保つことで、ログサーバで保存されるログの順序の入れ替わりが無いことを証明できる。一方で、ログの送信過程で独自ログシステム、ログサーバおよび通信路における障害の発生を想定しておく必要がある。障害の影響で、直前の送信ログのハッシュ値ずれもしくは、通信路における送信ログの喪失といった状況が発生すると考えられる。ログサーバにおいて、1 つのログでもハッシュ値検証に失敗すると、その後のハッシュ値の連鎖構造が維持できなくなる。そこで本手法では、ログの再送機能を設けることでハッシュ値の再検証をできるようにする。



図 5.3 ログデータの連鎖構造

再送処理

独自ログシステムからログサーバにログを送信する過程で、何らかの障害が発生する可能性がある。そこで、想定される障害に対応できる独自保全システムを構築することが求められる。想定される主な障害例として、独自保全システムの通信路で発生する障害が挙げられる。通信路の障害発生により、独自ログシステムはログをログサーバに適切に送信できない可能性がある。また、5.4.2 に述べたように、ハッシュ値検証機能において、何らかの理由でハッシュ値の連鎖構造にずれが生

じた場合、それ以降ログサーバに届いたログのハッシュ値検証に失敗する。そこで本手法では、独自ログシステムに再送機能を設ける。ログの再送機能により、何らかの理由でログサーバに対しログの送信に失敗しても、ログサーバで確実に記録されるまでログを送信できる。再送処理は、ログサーバから独自ログシステムへの確認応答によって処理を行うかどうか決定する。

5.5 難読化への対応

本研究における「難読化への対応」は、3.5 節の表 3.1 のような PowerShell スクリプトの難読化手法を組み込んだ攻撃に対し、難読化解除されたログの保全を目的とする。3.5.4 項で述べたように、難読化された (.NET) クラス・メソッドを用いた実行に対し、標準ログシステムにおいて難読化解除したログを取得できない。そのため本手法では、難読化された (.NET) クラス・メソッドの難読化解除及びログ保全を行う。

図 5.4 に示すように、PowerShell はコマンドあるいはスクリプトの入力に対し、AST (抽象構文木) に変換したものを Script Command Processor に渡す。Script Command Processor では、主にコンパイラが AST から Expression Tree への変換を行う。Expression Tree とは、コードをツリー状のデータ構造で表現したものである。Expression Tree への変換時に、変数や (.NET) クラス・メソッドの呼び出しが行われる。ユーザ入力コマンドまたはスクリプトが難読化されている場合においても、このとき呼び出される変数や (.NET) クラス・メソッドは難読化が解除されたものとなる。そこで本手法では、Script Command Processor での (.NET) クラス・メソッドの呼び出し処理をフックすることで、難読化された (.NET) クラス・メソッドの実行に関する難読化解除後のログを保全する。

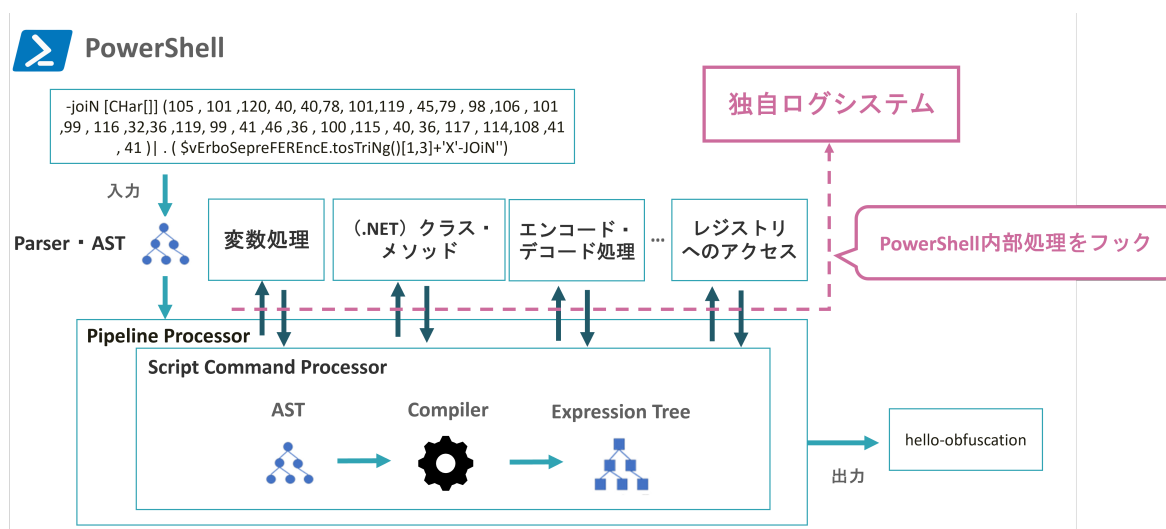


図 5.4 PowerShell 内部処理のフック

第 6 章

実装

6.1 実装環境

PowerShell に組み込む独自ログシステムおよびログサーバは，表 6.2 で示すような環境で実装する．

表 6.1 独自ログシステムの実装環境

種別	内容
OS	Windows 10 Education build 2004
PowerShell	7.3.0
.NET	7.0.0
C#	11.0

表 6.2 ログサーバの実装環境

種別	内容
OS	Ubuntu 20.04.4 LTS
Python	3.8.10

2.3.2 項で述べたように，2023 年 1 月時点の最新版として Windows PowerShell 5.1 および PowerShell 7.3 を利用できる．オープンソースとなった PowerShell 7 以降では，自身で PowerShell のソースファイルを編集し，ビルドを行うことで [22]，カスタムした PowerShell を使用できる．本手法では，独自ログシステムを PowerShell 内部処理系内でビルドして組み込むため，オープンソースである PowerShell 7.3 を用いることにした．

6.2 独自ログシステムの実装

6.2.1 ログサーバとのログ送信プロトコル

ログサーバにおけるログの署名およびハッシュ値検証を行うために、独自ログシステムとログサーバ間でのログ送信プロトコルを定める必要がある。本提案手法では、独自ログシステムからログサーバに送信するログパケットに図 6.1 に示すような情報を付加する。（付加した情報も含めたログを、「送信ログ」と呼ぶこととする）

() 内はサイズ (Byte)

ログデータ (可変長)	ログデータの署名 (172)	直前ログデータの ハッシュ値 (64)	ログ番号 (5)	PowerShell プロセス番号 (5)	デバイスID (36)	タイムスタンプ (32)	パケット 識別番号 (4)
----------------	-------------------	---------------------------	-------------	-----------------------------	----------------	-----------------	---------------------

図 6.1 送信ログのフォーマット

UDP 通信

本手法では、PowerShell 内部に独自ログシステムを組み込むという制約上、「UDP がコネクションレスで軽量である」、「独自ログシステムの完全性よりも PowerShell 実行処理の可用性を優先する」という理由により TCP ではなく UDP を選択した。

独自ログシステムは PowerShell 内部の実行処理系内に組み込まれるため、PowerShell 本体の処理に極力影響を与えないようにすることが望ましい。TCP のようなコネクション型のプロトコルであれば、N 個の PowerShell プロセスに対し、ログサーバと N 個のコネクションを保持し続けることになる。ユーザは Windows システム上において、複数の PowerShell プロセスを起動できる。そのため、プロセスが増えると、PowerShell プロセスを起動しているマシンやログサーバに加えて、ネットワーク環境に大きな負荷がかかると想定される。

ログを確実に保全するという点において、独自ログシステムの完全性は重要である。独自ログシステムからログサーバにログを確実に送信するためには、信頼性のある通信が保証される TCP を用いることが望ましい。一方で、TCP は UDP よりもオーバーヘッドが大きい。独自ログシステムからログサーバへのログの保全処理期間は、PowerShell 本体の実行処理が中断される。独自ログシステムを PowerShell に組み込むことにより、PowerShell 本体の実行処理性能の大幅な低下は避けるべきだと考える。そのため、本研究では独自ログシステムの完全性よりも PowerShell 実行処理の可用性を優先する。そこで、独自ログシステムとログサーバの通信に TCP ではなく UDP を用いることにした。一方で、UDP では送信したパケットが相手に届いたかどうか送信元では確認できない。さらに、送信した順番の保証がないといった信頼性の面で懸念が残る。そのため、独自ログシステム（ログサーバも含む）にログデータの署名や直前ハッシュ値の検証および再送処理の機能を組み込むことで、UDP 通信においても正確なログ送信を実現する。

ログデータの署名

独自ログシステムは、取得した PowerShell ログに対する署名を生成する。署名の生成には、.NET ライブラリに用意されている RSAPKCS1SignatureFormatter クラスの CreateSignature メソッドを利用した [23]。使用するハッシュアルゴリズムは SHA256 で、署名に必要な鍵は 6.2.2 項で述べる鍵交換時に生成した秘密鍵を用いる。

直前ログデータのハッシュ値

独自ログシステムでは、N 番目のログデータに対し、直前の N-1 番目のログのハッシュ値を付加して送信する。ハッシュ値の生成には、.NET ライブラリに用意されている HashAlgorithm クラスの ComputeHash メソッドを利用した。使用するハッシュアルゴリズムは SHA256 とした。

ログ番号

独自ログシステムで取得するログに、ログ番号としてシーケンス番号を付加する。

PowerShell プロセス番号およびデバイス ID

PowerShell は同一デバイス上で複数起動できるため、それぞれの PowerShell に組み込まれた独自ログシステムを識別する必要がある。つまり、ログサーバは、受け取ったログが同一デバイス上のどの独自ログシステムのものかを識別する必要がある。そこで、PowerShell のプロセス番号とその PowerShell を起動しているデバイス ID を送信ログデータに付加する。

タイムスタンプ

独自ログシステムがログを取得した時刻をタイムスタンプとして、送信ログに付加する。タイムスタンプの取得には、.NET ライブラリに用意されている DateTime.UtcNow プロパティを利用した。また、日付と時刻のフォーマットとして、ISO 8601 の基本形式を用いる。

パケット識別番号

本手法における独自ログシステムとログサーバとの通信の種類には、「ログ送信」、「ログ再送」、「鍵交換」、「確認応答（正常処理）」、「確認応答（再送要求）」がある。独自ログシステムおよびログサーバは、通信の種類をログに付加されたパケット識別番号によって識別する。また、パケット識別番号は 6.2.4 項で述べる分割送信にも用いる。パケット識別番号ヘッダの上位 2 バイトを、通信の種類の識別のために、下位 2 バイトを分割送信のために用いる。表 6.3 にあるように、独自ログシステムおよびログサーバは受信したパケットの識別番号に応じて処理を行う。

表 6.3 パケット識別番号上位 2 バイトと通信の種類

バイト列	通信内容	通信方向
01	ログ送信	独自ログシステム → ログサーバ
02	鍵交換	独自ログシステム → ログサーバ
03	ログ再送	独自ログシステム → ログサーバ
11	確認応答（正常処理）	ログサーバ → 独自ログシステム
12	鍵交換	ログサーバ → 独自ログシステム
13	確認応答（再送要求）	ログサーバ → 独自ログシステム

6.2.2 ログサーバとの鍵交換

5.4.2 項で述べたように、独自ログシステムは送信ログに署名をつけてログサーバに送信する。ログの署名は、対象となるログデータのハッシュ値をとったものに対し、秘密鍵で暗号化することで生成できる。

また、独自ログシステムとログサーバ間の通信路において、第三者（攻撃者）に平文のログパケットを盗聴されると、ログ送信の機密性が失われる。TCP 通信においては SSL/TLS の暗号化プロトコルを用いれば、容易に暗号化通信が可能である。一方で本手法では、UDP 通信を用いるため、独自ログシステム側で送信ログおよび確認応答に対する暗号化機能を組み込む必要がある。ログの署名および送信ログの暗号化には、ログサーバとの間で鍵ペア（秘密鍵-公開鍵）を共有しておく必要がある。そこで独自ログシステムでは、PowerShell を起動してコンソールセッションを開始する前に、以下の流れでログサーバと鍵交換を行う。鍵交換および送信ログの暗号化の流れを図 6.2 に示す。

1. 独自ログシステムで秘密鍵 sk と公開鍵 pk のペアを生成。
2. 独自ログシステムからログサーバへ公開鍵 pk を送信。
3. ログサーバは共通鍵ペア k, k' を生成。
4. ログサーバは共通鍵 k' を受け取った公開鍵 pk で暗号化。
5. ログサーバは暗号化した共通鍵 k' ($E_{pk}k'$ とする) を独自ログシステムに送信。
6. 独自ログシステムは受け取った $E_{pk}k'$ を秘密鍵 sk で復号し、共通鍵 k' を取得。

鍵交換成功後、独自ログシステムでは秘密鍵 sk をを用いて、ログの署名を生成する。ログサーバでは、独自ログシステムから受け取った公開鍵 pk をを用いて署名を検証する。また、独自ログシステムでは共通鍵 k' をを用いて送信ログの暗号化を行う。ログサーバでは、保持している共通鍵 k をを用いて暗号化された送信ログを復号する。

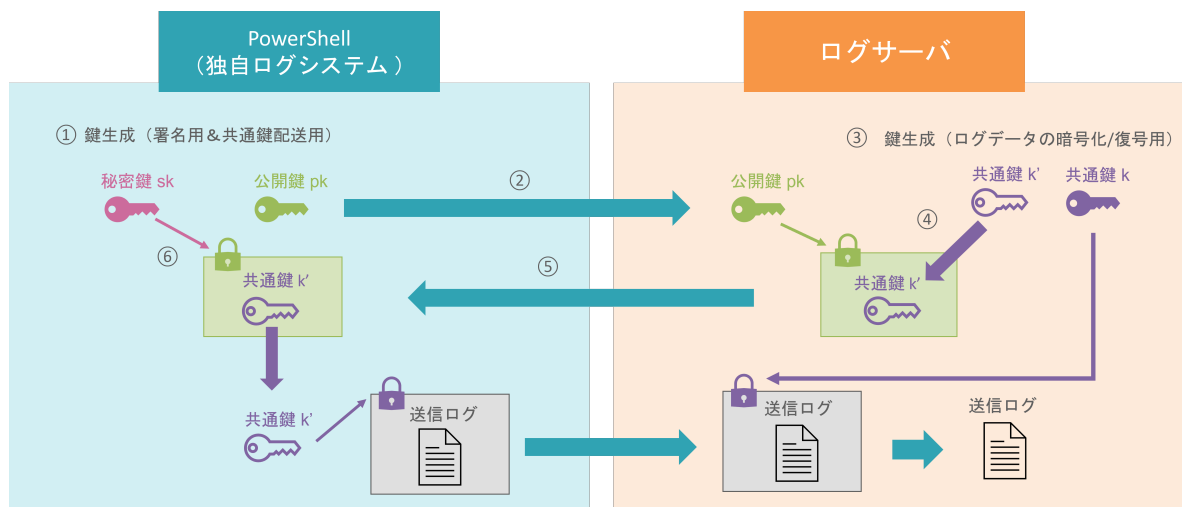


図 6.2 鍵交換の流れ

6.2.3 再送処理

再送処理が発生する条件は以下の通りである。

- 独自ログシステムからログサーバに送信ログが到達。
- ログサーバで署名またはハッシュ値検証に失敗。
- ログサーバから独自ログシステムに対し、確認応答が未達。
- 独自ログシステムとログサーバの通信路で障害が発生し、パケットロスの発生。
- ログサーバがダウンした場合。

ログサーバは、独自ログシステムからの送信ログを受け取ると、独自ログシステムに対し確認応答を送信する。ログサーバは、送信ログの処理結果に応じて、確認応答パケットの識別番号上位 2 バイトを表 6.3 のように設定する。独自ログシステムは、受け取った確認応答パケットの識別番号上位 2 バイトを確認して、再送処理を行うかどうか決定する。確認応答パケットには、再送処理の有無に関わらず、ログサーバが処理したログ番号が付加されている。独自ログシステムは、再送処理が必要な場合、確認応答に付加されたログ番号に応じて、再送を行う。ただし、6.3.2 項で述べるハッシュ値検証の仕様上、再送対象となるログデータが一つではなく複数存在する可能性がある。そのため、ログサーバは、再送対象となるログデータを列挙した再送要求リストを確認応答に付加し、独自ログシステムに送信する。ここで、独自ログシステムは再送処理を行うために、ローカル内のストレージにログデータを一時的に保存しておく。独自ログシステムはログサーバから送られた再送要求リストのログと同じローカルログデータをログ番号により検索し、再送する。ログサーバで送信ログが正常に記録できれば、確認応答に付加されたログ番号と同じローカル内のログを削除する。記録されるログの量に比例してローカルストレージの使用量も増えていくため、一時

保存用のローカルログデータを削除する方が望ましい。また、ログサーバからの確認応答が返ってこない状況も考えられる。その場合、ログの再送を数回実施し、それでも応答がなければサーバがダウンしていると判断する。

ただし本手法では、ローカルストレージに一時保存しているため、攻撃者が保存場所情報および管理者権限を取得している場合、ログの削除や改ざんの可能性が考えられる。この場合、独自ログシステムにおける通常のログ送信への影響は無いが、再送処理が正常にできない可能性が考えられる。

6.2.4 分割送信

PowerShell で実行できるコマンドおよびスクリプトのサイズに制限がない。そのため、データサイズが非常に大きいコマンドやスクリプトを実行することで、生成されるログのデータサイズも大きくなる。UDP において、ログを正確に送信するためには、送信するログのデータサイズを極力小さくする方が望ましい。そこで、UDP はパケット分割機能を持たないため、独自ログシステムで分割機能を実装した。独自ログシステムからログサーバへのログ送信において、送信ログのサイズを最大 500 バイトとし、それ以上であればログデータを 500 バイトごとに分割し送信する。分割送信の場合、送信ログのパケット識別番号ヘッダの上位 3, 4 バイト目に分割シーケンス番号を設定する。ログサーバは、パケット識別番号ヘッダの上位 3, 4 バイト目にある分割シーケンス番号に従って、分割されたログデータを結合して保存する。

6.2.5 PowerShell 内部処理のフック

本研究では、(.NET) クラス・メソッドの実行に関わる PowerShell 内部処理のフックを目指す。 .NET では、InvokeMember メソッドを用いて、クラス・メソッドの呼び出しを行う。 .NET を基盤とする PowerShell においても、クラス・メソッドの呼び出しは InvokeMember メソッドを用いる。本手法では、PowerShell/src/System.Management.Automation/engine/runtime/Binding/binders.cs [24] の FallbackInvokeMember メソッドにおける、クラス・メソッドの呼び出しをフックする。FallbackInvokeMember メソッドをフックすることで、呼び出すクラス・メソッドに加えて、メソッドのパラメータも取得できる。また、ここで処理されるクラス・メソッドおよびパラメータは難読化の影響を受けない。

6.3 ログサーバの機能

6.3.1 ログの記録

独自ログシステムから受け取った送信ログをログサーバに JSON 形式で記録する例を図 6.3 に示す。ログサーバにおいて、ログの内容に加え、署名とハッシュ値検証の結果を記録する。ログサーバは、署名とハッシュ値検証の両方が成功した場合に Status (ステータス) キーを「VERIFIED」

とし、ログを保存する。このステータスに関しては次項で詳しく述べる。

```
"51": {
  "Number": "00051",
  "TimeStamp": "2022-12-10T17:09:51.764195+09:00",
  "DeviceId": "A0A83611-74C5-49F5-8127-73844D3443B1",
  "ProcessId": "10008",
  "LogData": "Set-ExecutionPolicy Restricted -Scope Process",
  "Hash": "eeccb00e2a13c1644aae5f1f23a6cdcebf63321fedbc11edd2ca5a7df346a3cf",
  "Signature": "X99mRTQLKZSrv45X5JYHZdJaApMsao99/VWVnY3fK50rdbzN3c6sgol1prXjHIs9UCY9t
IdGG6U/WJi0PKaMZ++VLpswZ7KpKQFamHRk6gMCKKYImXDU7qVBEmkZ/FTN4Yc2JiU/wn7JCjdnFGvgJT0ZXdp
  "Sign_Verification": true,
  "PreLogHash": "52c27973088975e3165c63e008e0b1d65735f6508402d948e03998bffe907211",
  "Hash_Verification": true,
  "Status": "VERIFIED"
},
"52": {
  "Number": "00052",
  "TimeStamp": "2022-12-10T17:10:05.181515+09:00",
  "DeviceId": "A0A83611-74C5-49F5-8127-73844D3443B1",
  "ProcessId": "10008",
  "LogData": "Get-Verb",
  "Hash": "4eeb22022be5bdc9c802cf3a47b46857f81d7be6c5e6f8ebad917562460758ff",
  "Signature": "XBzBfNQgsr360l6YP8vK8pLSDcSsJky/xPEcePpTmfsgtdh0XIfpHVbDDtc+h41cCXED+
um5mp01Lnryq9oRq5YnS2Gmd+q52cI6Toq9SCK4d2eESYw8XfoGsDwAJnntFdRZkeMgKbWJEn9BR5ebk7byciHb
  "Sign_Verification": true,
  "PreLogHash": "eeccb00e2a13c1644aae5f1f23a6cdcebf63321fedbc11edd2ca5a7df346a3cf",
  "Hash_Verification": true,
  "Status": "VERIFIED"
},
```

図 6.3 ログサーバにおけるログ保全の記録例

6.3.2 署名とハッシュ値検証

ログサーバは、独自ログシステムからの送信ログを受け取った後、はじめに署名とハッシュ値検証を行う。ハッシュ値検証では、直前のログデータのハッシュ（以下、「直前ログハッシュ」とする）を参照するため、ログサーバに直前のログデータが記録されていなければならない。とある送信ログ（N 番目のログ）で再送処理が発生した場合、そのログの再送が完了するまでに次の送信ログ（N+1 番目のログ）がログサーバに到着する可能性がある。その場合、N 番目のログが存在しないため N+1 番目のログのハッシュ値検証は必ず失敗してしまう。そこで、ログサーバにおいて独自ログシステムから受け取った送信ログに「ステータス」を付与し管理することで、ログの到着順序に依存せずハッシュ値検証ができるようにする。再送処理を考慮した署名とハッシュ値検証のフローを図 6.4 に示す。

H_s : サーバに保存された直前 (N-1 番目) のログハッシュ

H_p : PSから受信したログデータ (N 番目) に付加された直前ログハッシュ

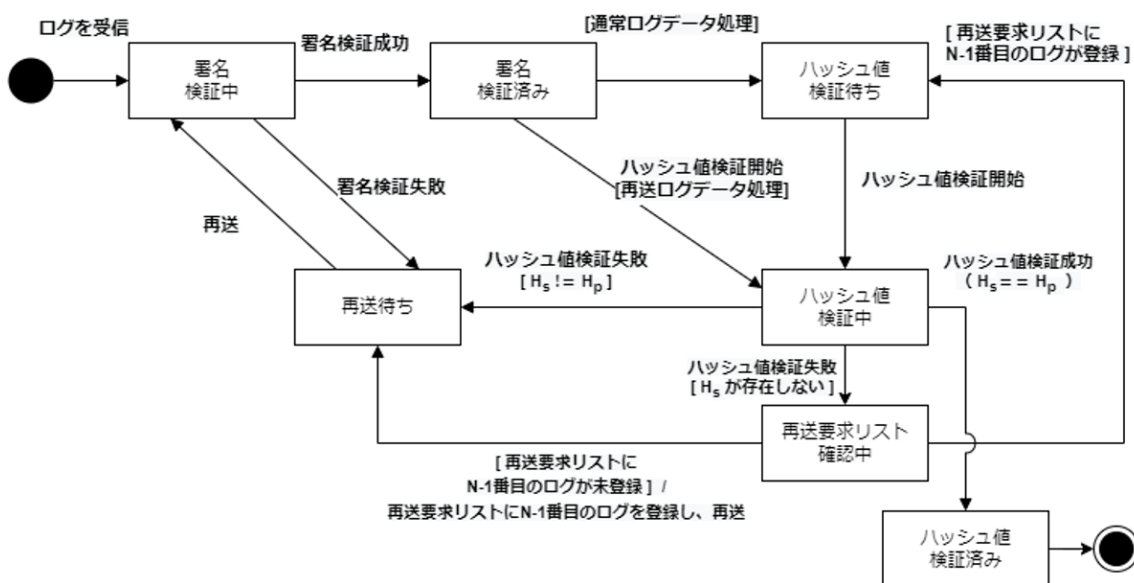


図 6.4 ログサーバにおける署名とハッシュ値検証のフロー

ログサーバでは、署名検証成功後にハッシュ値検証を行う。署名検証に失敗した場合は、ログサーバは直ちに再送要求を独自ログシステムに確認応答として送信する。再送処理を想定したハッシュ値検証処理の例を図 6.5 に示す。

この例では以下の流れで、ログサーバがハッシュ値検証の処理を行う。この例において、独自ログシステムからログ 3 を送信したが、何らかの理由でログサーバにおけるログ 3 のハッシュ値検証

に失敗した状況を想定する。また、ログ3の再送到達までにログ4とログ5がログサーバに送信されているものとする。

1. ログ3で再送が発生した場合、ログ3のステータスは「再送待ち」となる。
2. 再送までの間、独自ログシステムから送信された通常ログ4とログ5は、署名検証を行った後にハッシュ値検証は行わず、ステータスを「検証待ち」にする。ハッシュ値の検証を行わない理由は、ログ3のハッシュ値の検証が成功しておらず、それ以降のログ4とログ5は必ず検証に失敗するためである。
3. ログ3が再送されれば、ログ3のハッシュ値検証を行った後に、ステータスが「検証待ち」のログ4とログ5のハッシュ値検証を順次再開する。
4. ハッシュ値の検証に成功すれば、ログ3とログ4およびログ5のステータスを「検証成功 (VERIFIED)」に更新する。

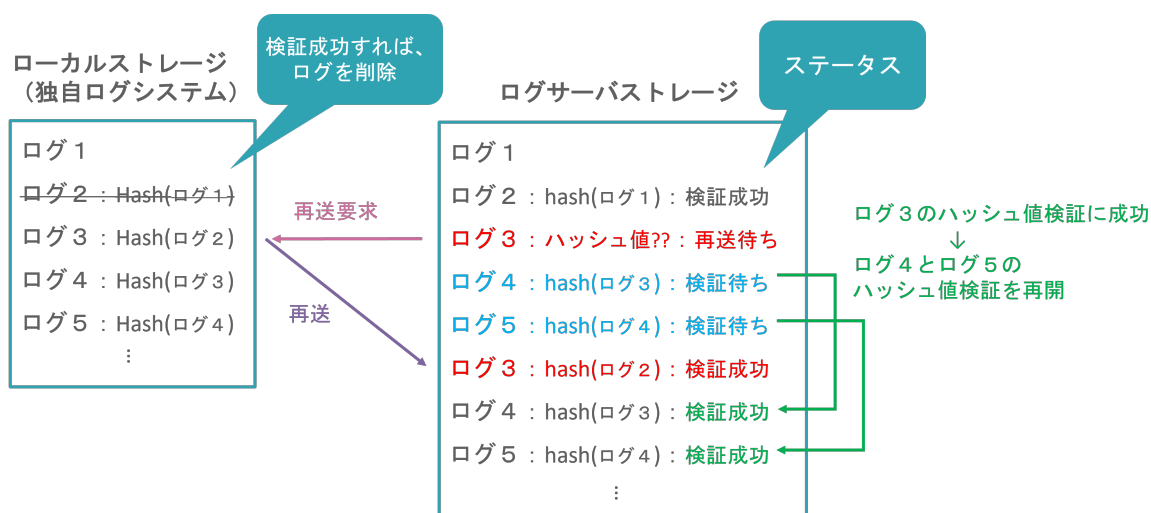


図 6.5 ハッシュ値検証と再送処理

第 7 章

評価

7.1 概要

本研究で提案した独自保全システムを用いて、アンチ・フォレンジック適用下でも確実にログを保全できるか検証する。本実験において想定するアンチ・フォレンジックは、3.5 節で挙げた「PowerShell ログシステムの無効化」、「PowerShell ログの改ざん・削除」、「PowerShell スクリプトの難読化」とする。実装した独自保全システムにおけるログ保全の可否を検証するために、独自ログシステムを組み込んでカスタマイズした PowerShell をユーザが実際に操作したときのログの保全状況を調査する。

7.2 独自ログシステムの性能

PowerShell 本体の実行性能を極力下げずに、独自ログシステムを PowerShell に組み込むことが望ましい。そこで、PowerShell 内部に独自ログシステムを組み込むことにより発生するオーバーヘッドを検証した。表 7.1 において、独自ログシステムを組み込み有無の PowerShell それぞれで「Write-Output "hello world!!"」コマンドの実行時間を計測した。表 7.2 に 50 回の平均実行時間示す。

表 7.1 実行環境

種別	内容
CPU	Intel Core i7-10700F @ 2.90GHz
OS	Windows 10 Education build 2004
Memory	8GB 3200MHz DDR4
PowerShell	7.3.0 (.NET 7.0)

表 7.2 独自ログシステム組み込み有無による実行時間

独自ログシステムの組み込み	平均実行時間
無し	1.2381 ms
有り	1.6793 ms

表 7.2 の通り，独自ログシステムを PowerShell 内部に組み込むことで発生するオーバーヘッドは約 0.44ms である．

7.3 独自保全システムにおけるログ保全の可否

攻撃者による PowerShell を用いたファイルレス攻撃を想定し，カスタム PowerShell においてもログが保全できているか検証する．また，3.5 節で述べたようなアンチ・フォレンジックの「PowerShell ログシステムの無効化」および「PowerShell ログの改ざん・削除」が搭載された攻撃においても，確実にログを保全できるか検証する．

7.3.1 想定する状況

本実験では，PowerShell を用いたファイルレス攻撃のシナリオとして，図 2.5 に示すような攻撃シナリオを用いる．攻撃者が標的 PC と C&C サーバとの遠隔操作通信を確立し，C&C サーバから遠隔操作で標的 PC の PowerShell 上で悪意のあるコマンドやスクリプトを実行する状況を想定する．また，攻撃過程において 3.5 項で述べたようなアンチ・フォレンジック手法が組み込まれている状況を想定する．

7.3.2 実験方法

本実験では，被験者を攻撃者と見立てる．また，被験者が PowerShell を操作する状況を，攻撃者が C&C サーバから遠隔操作で悪意のある PowerShell コマンドやスクリプトを実行する状況として想定する．実験において，管理者と被験者は以下の操作を行った．

1. 管理者が，Windows システム内に残っているログデータおよびコマンド実行履歴を削除する．（実験環境の初期化）
2. 管理者が，ログサーバを起動する．
3. 被験者が，独自ログシステム搭載のカスタム PowerShell を起動する．
4. 被験者が，カスタム PowerShell の操作を開始する．
5. 被験者が，カスタム PowerShell を任意の期間で操作する．（操作 1）
6. 被験者が，ローカル内標準の PowerShell ログ（Event ID：4104 の Windows イベントロ

グ)を削除する。(操作2)

7. 被験者が、カスタム PowerShell を任意の期間で操作する。(操作3)
8. 被験者が、標準の PowerShell ログシステムを無効化する。(操作4)
9. 被験者が、カスタム PowerShell を任意の期間で操作する。(操作5)
10. 被験者が、カスタム PowerShell の操作を終了する。
11. 管理者が、ローカル内標準の PowerShell ログとコマンド実行履歴(Console-Host.history.txt)を収集する。

被験者はカスタム PowerShell の操作期間内で、PowerShell ログの削除(操作2)と PowerShell ログシステムの無効化(操作4)を実行した。また、被験者によるコマンド実行履歴の削除は禁止とした。これは、7.3.3 項で述べるように、評価において、ログの保全状況の検証にコマンド実行履歴を利用するためである。また、本システムでは再送処理を記録する機能を搭載しており、本実験における再送処理の発生を確認できる。

7.3.3 評価方法

標準ログシステムおよび独自ログシステムにおける、ログ保全状況を検証する。ユーザが実行したコマンドおよびスクリプトの履歴は、PowerShell のコマンド履歴としてファイル(Console.history.txt)に記録される。コマンド履歴は、ログシステムと無関係であるため、被験者が実験中に適用するアンチ・フォレンジック手法の影響を受けない。そのため、実験中に被験者が実行したすべてのコマンドおよびスクリプトを確認できる。コマンド履歴に対し、標準ログシステムおよび独自ログシステムに記録されたログを比較して、ログの保全状況の評価する。独自ログシステムにおいては、記録されたログの署名とハッシュ値検証に成功し、ステータスが「VERIFIED」であるものを保全できたログとする。また、本システムに搭載されている再送処理の記録を確認し、再送処理が発生した場合でもログが保全できているか確認する。図 7.1 に、ログ保全状況の検証の例を示す。

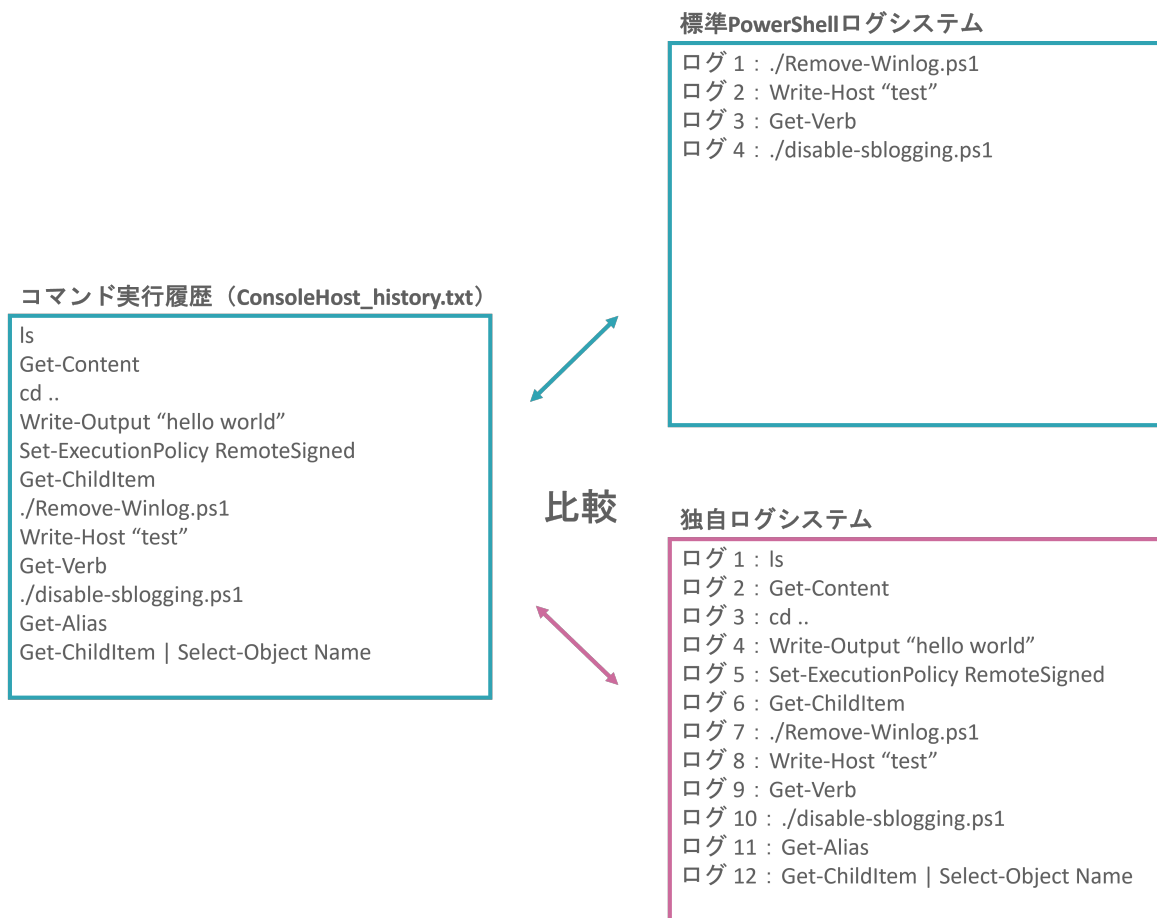


図 7.1 ログ保全状況の検証例

7.3.4 結果

被験者 10 名が使用した PowerShell の標準ログシステムおよび独自ログシステムそれぞれにおける、ログ保全結果を図 7.3 に示す。標準ログシステムにおいて、操作 1 および操作 5 に対するログが保全できていなかった。今回実装した独自ログシステムは、ユーザが実行したすべてのコマンドおよびスクリプトの実行ログを保全できた。また、再送処理の発生記録を確認すると、被験者 B と被験者 H において再送処理が発生していた。

表 7.3 標準ログシステムと独自ログシステムのログ保全結果

	標準ログシステム	独自ログシステム	実行総数
被験者 A	△ (39) *	○ (1108)	1108
被験者 B	△ (176)	○ (373)	373
被験者 C	△ (230)	○ (553)	553
被験者 D	△ (69)	○ (209)	209
被験者 E	△ (233)	○ (367)	367
被験者 F	△ (54)	○ (200)	200
被験者 G	△ (91)	○ (214)	214
被験者 H	△ (29)	○ (276)	276
被験者 I	△ (167)	○ (386)	386
被験者 J	× (0)	○ (237)	237

* () 内はユーザが実行したコマンドおよびスクリプトのうち、保全できたログ数を示す。

○：ユーザが実行したコマンドおよびスクリプトのログをすべて保全できている。

△：ユーザが実行したコマンドおよびスクリプトのログを一部保全できている。

×：ユーザが実行したコマンドおよびスクリプトのログを 1 つも保全できていない。

7.4 独自ログシステムの難読化解除

難読化済みのクラス・メソッドの実行に対し、独自ログシステムで難読化解除されたログが保全できるか検証した。本実験では、ペネトレーションテストツールの Atomic Red Team [25] が公開しているスクリプトのうち、プロセスインジェクションを目的としたスクリプト [26]（以下、「テストスクリプト」と呼ぶ）を用いた。テストスクリプトでは、「WriteProcessMemory」や「VirtualAllocEx」をはじめとする Windows のネイティブ API を用いるためのクラス・メソッドを定義している。テストスクリプトで定義されているクラス・メソッドを表 7.4 に示す。独自ログシステムを組み込んだカスタム PowerShell で、表 3.1 の手法を用いて難読化したテストスクリプトを実行した。

難読化したテストスクリプトの実行後、独自ログシステムが表 7.4 のすべてのクラス・メソッドの難読化解除後のログを保全できていることを確認した。

表 7.4 テストスクリプトで定義・使用されるクラス・メソッド

クラス	メソッド
Hollow	OpenProcess
	WriteProcessMemory
	ReadProcessMemory
	VirtualAllocEx
	VirtualProtectEx
	NtOpenFile
	NtCreateSection
	NtCreateProcessEx
	NtQueryInformationProcess
	RtlCreateProcessParametersEx
	NtCreateThreadEx

第 8 章

考察と今後の課題

8.1 本評価に対する考察

8.1.1 独自ログシステムの性能に関する考察

本実験では、PowerShell に独自ログシステムを組み込むことによる、PowerShell 本来の実行性能の影響を検証した。表 7.2 で示したように、独自ログシステム組み込みの有無において PowerShell での平均実行時間の差が、約 0.45ms であった。つまり、独自ログシステムを PowerShell 内部に組み込むことで発生するオーバーヘッドが、約 0.45ms であると言える。このことから、ユーザが PowerShell を通常利用する上で、独自ログシステムの組み込みによる PowerShell 本体の実行性能の低下はほとんど無いと言える。

8.1.2 独自保全システムにおけるログ保全の可否に関する考察

本実験では、典型的なファイルレス攻撃を想定し、攻撃者と見立てた被験者が独自ログシステム有無それぞれの PowerShell を操作した。また、被験者は任意の操作に加えて、7.3.2 項の実験方法に従って、PowerShell ログシステムの無効化および PowerShell ログの削除を実施した。被験者 10 名の標準ログシステムおよび独自ログシステムそれぞれのログ保全結果は、図 7.3 に示す通りとなった。本評価において、5.3 節で定めた、ログを確実に保全するためのすべての要件を満たすことができたと言える。このことから、独自ログシステムを組み込んだカスタム PowerShell において、ユーザの実行ログを確実に保全できたと言える。さらに、3.5 節で挙げたアンチ・フォレンジック手法の「PowerShell ログシステムの無効化」および「PowerShell ログの削除」の適用下においても、ログを保全できたと言える。

また、被験者 J において標準ログシステムでは、ユーザが実行したコマンドおよびスクリプトのログを 1 つも保全できなかった。これは、7.3.2 項の実験方法において、被験者が以下のいずれかの操作を行ったためであると考えられる。

- 操作 2 の完了後、操作 3 から操作 5 を行わずに PowerShell の操作終了。

- 操作 2 の完了後，操作 3 を未実施．
- PowerShell の操作開始後，操作 1 から操作 3 を行わずに操作 4 を実施．

本実験の結果より，本システムの再送処理記録において，被験者 B と被験者 H では再送処理が発生していた．再送処理に失敗している場合，6.3.2 項に述べたように，再送処理要求されているログを含め，それ以降のログのハッシュ値検証が失敗する．本評価においては，ログサーバに記録されているログのステータスが「VERIFIED」であれば，独自保全システムにおいてログ保全に成功していると判断する．つまり，再送処理に失敗している場合，再送対象のログを含めそれ以降のログのステータスが「VERIFIED」とならず，ログ保全に失敗していると判断される．一方で，表 7.2 において，実験中に再送処理が発生した被験者 B と被験者 H の独自ログシステムのログ保全が正常に行われている．したがって，本システムにおける再送処理が正常に実施されたと言える．

本手法で実装した独自ログシステムは，PowerShell 内部に組み込むため，PowerShell のプロセス上で機能する．そのため，PowerShell 本体の実行に影響を与えず，外部からの独自ログシステムへの干渉は困難である．また，標準 PowerShell は，汎用的なユーザ使用を想定している都合上，ログシステムの有効化・無効化の設定機能を備えている．ログシステムの有効化・無効化の設定機能は，攻撃者に狙われやすく，3.5.1 項の PowerShell ログシステムの無効化や，管理者権限による設定変更の影響を受けやすい．そこで，独自ログシステムは，ユーザによる独自ログシステムの有効化・無効化の設定機能を持たず，強制的にログを記録する．さらに，独自ログシステムは，ログをローカル外のログサーバに送信することで，ログの改ざん・削除から保護することができる．本研究の評価において，PowerShell を用いたファイルレス攻撃として，攻撃者による C&C サーバから標的 PC に対する PowerShell コマンドやスクリプトの遠隔実行のログを確実に保全できると言える．

8.2 今後の課題

8.2.1 独自保全システムの実用性の向上

本研究で実装した独自保全システムの再送処理は，主に障害発生時に実行される．本手法における再送処理の実行には，障害発生から復帰ができた場合に限るという前提条件がある．例えば，ネットワーク通信障害が発生し，その後復旧できない場合，独自ログシステムからログサーバにログを保全できない．また，本手法の再送処理では，ログをローカルマシン内に一時的に保存する．そのため，攻撃者によって，一時保存されたログを改ざんもしくは削除された場合，再送ができないという課題がある．そこで，再送処理用にローカルマシン内に一時保存するログも何らかの保護をする必要がある．

また，本手法で実装したカスタム PowerShell は，OSS の PowerShell 7 に独自ログシステムを組み込んだものである．2.3.2 項で述べたように，PowerShell 7 を利用するためには追加でインストールしなければならない．つまり，本手法において，独自ログシステムを組み込んだカスタム PowerShell を Windows PC にインストールする必要がある．さらに現状，インストールした

カスタム PowerShell は、Windows 標準搭載の Windows PowerShell と共存することになる。そこで攻撃者は、カスタム PowerShell ではなく、Windows PowerShell を使用する可能性がある。Windows PowerShell に備わる標準ログシステムでは、本研究で想定しているようなアンチ・フォレンジック手法を組み込んだ攻撃に対し、攻撃に関わる PowerShell ログを保全できない。この対策として、システム管理者は、グループポリシーの設定により Windows PowerShell を無効化もしくは、Windows PowerShell 実行ファイル (powershell.exe) を削除し、カスタム PowerShell の使用を強制する方法が挙げられる。一方で、2.3.2 項で述べたように、将来的に Windows PowerShell から PowerShell 7 への完全な移行が想定され、これにより、上記の Windows PowerShell との併用問題の解決が期待される。

さらに、PowerShell 7 は Windows PowerShell の基本部分を引き継いでいるが、一部のコマンドレットや機能が利用できない、そのため、Windows PowerShell で実行できていたスクリプトが、カスタム PowerShell で実行できる保証はない。現状、本提案システムは、PowerShell ログを確実に保全するという点において有効である。一方で、Windows PowerShell から PowerShell 7 への移行というような、本手法の実運用における課題を解決する必要がある。

8.2.2 難読化解除機能の拡充

本研究では、PowerShell スクリプトの難読化解除の課題に対し、(.NET) クラス・メソッドの難読化解除を実現した。一方で、この結果において、PowerShell スクリプトの難読化解除を実現したとは言えない。PowerShell スクリプトにおいては、(.NET) クラス・メソッドの他にも、関数や変数およびコマンドレットのような機能を利用できる。現状、PowerShell スクリプトの完全な難読化解除を実現することは困難である。しかし、フック可能な PowerShell 内部処理が増えることで、PowerShell スクリプトで使用されている機能の解析に大きく役立つと考える。

第 9 章

おわりに

本研究では、PowerShell 内部に独自ログシステムを組み込み、直接ローカル外のログサーバにログを保全する手法を提案した。多くの攻撃者は、ファイルレス攻撃を実行する際に、PowerShell を利用する。さらに、攻撃者はデジタル・フォレンジック調査員の解析を妨害する目的とし、攻撃に関わるアーティファクトである PowerShell ログを極力残さないようにする。このような状況の中で、本研究で示したような、カスタマイズされた PowerShell を用いた保全システムを実用化し、インシデント調査に貢献できれば、本研究の本意である。

謝辞

本研究を進めるのにあたり，熱心なご指導を賜りました上原哲太郎教授には厚く御礼申し上げます．また，共同研究として数々のご助言をいただきましたデロイトトーマツサイバー合同会社様に深く感謝を申し上げます．最後に，実験のご協力ならびに，本論文の作成を進めるにあたり，多くのご意見やご指摘を頂きましたサイバーセキュリティ研究室の皆様に深く感謝致します．

参考文献

- [1] CIS (Center for Internet Security) . Only in Memory: Fileless Malware - An Elusive TTP. <https://www.morphisec.com/hubfs/2020\%20State\%20of\%20Endpoint\%20Security\%20Final.pdf>. [最終閲覧日:2023-1-14].
- [2] Kaspersky. Incident Response Analyst Report 2020. https://media.kasperskycontenthub.com/wp-content/uploads/sites/43/2020/08/06094905/Kaspersky_Incident-Response-Analyst_2020.pdf. [最終閲覧日:2023-1-14].
- [3] 総務省. 令和 4 年版 情報通信白書. <https://www.soumu.go.jp/johotsusintokei/whitepaper/ja/r04/pdf/01honpen.pdf>. [最終閲覧日:2023-1-14].
- [4] 独立行政法人情報処理推進機構. 情報セキュリティ 10 大脅威 2023. <https://www.ipa.go.jp/security/vuln/10threats2023.html>. [最終閲覧日:2023-1-26].
- [5] AhnLab. ハッカーの強力な武器、ファイルレス攻撃とは. https://jp.ahnlab.com/global/upload/download/asecreport/PressAhn_vol182.pdf. [最終閲覧日:2023-1-14].
- [6] Microsoft. PowerShell とは. <https://learn.microsoft.com/ja-jp/powershell/scripting/overview?view=powershell-7.3>. [最終閲覧日:2023-1-14].
- [7] Microsoft. PowerShell Core 6.0: Generally Available (GA) and Supported! <https://devblogs.microsoft.com/powershell/powershell-core-6-0-generally-available-ga-and-supported/>. [最終閲覧日:2023-1-14].
- [8] LOCKHEED MARTIN. THE CYBER KILL CHAIN. <https://www.lockheedmartin.com/en-us/capabilities/cyber/cyber-kill-chain.html>. [最終閲覧日:2023-1-14].
- [9] National Institute of Standards and Texhnology. Computer security incident handling guide. *NIST SP 800-61 Revision 2*, 2012.
- [10] デジタル・フォレンジック研究会. 証拠保全ガイドライン第 8 版. https://digitalforensic.jp/wp-content/uploads/2020/10/guideline_8thv1.12.pdf. [最終閲覧日:2023-1-10].
- [11] 安富潔, 上原哲太郎 (編著). 特定非営利活動法人デジタル・フォレンジック研究会: 基礎から学ぶデジタル・フォレンジック. 日科技連出版社, 2019.
- [12] National Institute of Standards and Texhnology. Guide to integrating forensic techniques into incident response. *NIST SP 800-86*, 2006.

- [13] National Institute of Standards and Texhnology. Guide to computer security log management. *NIST SP 800-92*, 2006.
- [14] Microsoft. about_Logging_Windows. https://learn.microsoft.com/ja-jp/powershell/module/microsoft.powershell.core/about/about_logging_windows?view=powershell-7.3. [最終閲覧日:2023-1-14].
- [15] Microsoft. about_Script_Blocks. https://learn.microsoft.com/ja-jp/powershell/module/microsoft.powershell.core/about/about_script_blocks?view=powershell-7.3. [最終閲覧日:2023-1-14].
- [16] Microsoft. Start-Transcript. <https://learn.microsoft.com/ja-jp/powershell/module/microsoft.powershell.host/start-transcript?view=powershell-7.3>. [最終閲覧日:2023-1-14].
- [17] Ryan Cobb. PowerShell ScriptBlock Logging Bypass. <https://cobbr.io/ScriptBlock-Logging-Bypass.html>. [最終閲覧日:2023-1-14].
- [18] LAC. PowerShell Empire を利用した標的型攻撃. https://www.lac.co.jp/lacwatch/people/20170807_001352.html. [最終閲覧日:2023-1-14].
- [19] Sophos. A new ransomware enters the fray: Epsilon Red. <https://news.sophos.com/en-us/2021/05/28/epsilonred/>. [最終閲覧日:2023-1-14].
- [20] 芦野佑樹, 佐々木良一. セキュリティデバイスとヒステリシス署名を用いたデジタルフォレンジックシステムの提案と評価. 情報処理学会論文誌, Vol. 49, No. 2, pp. 999 – 1009, 2008.
- [21] 高田哲司, 小池英樹. 逃げログ: 削除まで考慮にいたログ情報保護手法. 情報処理学会論文誌, Vol. 41, No. 3, pp. 823 – 831, 2000.
- [22] PowerShell Contributors. Build PowerShell on Windows for .NET Core. <https://github.com/PowerShell/PowerShell/blob/master/docs/building/windows-core.md>. [最終閲覧日:2023-1-14].
- [23] Microsoft. Cryptographic Signatures. <https://learn.microsoft.com/en-us/dotnet/standard/security/cryptographic-signatures>. [最終閲覧日:2023-1-14].
- [24] Microsofti PowerShell Team. PowerShell [Source code]. <https://github.com/PowerShell/PowerShell>. [最終閲覧日:2023-1-14].
- [25] Atomic Red Team. Explore Atomic Red Team. <https://atomicredteam.io/>. [最終閲覧日:2023-1-14].
- [26] Atomic Red Team. T1055.012 - Process Injection: Process Hollowing. <https://github.com/redcanaryco/atomic-red-team/blob/master/atomics/T1055.012/T1055.012.md>. [最終閲覧日:2023-1-14].